

C++ PROGRAMMING FOR EMBEDDED SYSTEMS & ROBOTICS

A Complete 24-Week Secondary School Manual for Absolute Beginners

CREATED & TAUGHT BY: Abdullahi Oladipupo Muwatta

FOUNDER: Algorise Tech Explorers (ATE)

WEBSITE: www.muwatta.com.ng

PROGRAM LENGTH: 24 Weeks, 96 Hours Total

LEVELS: 10 Core Levels + 8 Capstone Projects

TARGET AUDIENCE: Secondary School Students (Ages 14+)

PREREQUISITES: None - For Absolute Beginners

A Message from Your Teacher, Mr. Muwatta

Hello, my name is Abdullahi Oladipupo Muwatta. I founded Algorise Tech Explorers because I believe young people in Nigeria and Africa can master engineering skills at the highest level. This manual is written directly for you.

You might think that programming and robotics are only for people far away in other countries. You might think you need to be a genius to understand these things. Both of these thoughts are wrong. Everything in this manual is designed for someone with zero prior experience.

I have not written this as a textbook to memorise facts from. I have written it as a guide for learning by DOING. You will write real code. You will solder real circuits. You will build systems that sense what is happening around them, make decisions based on what they sense, and take action in the real world. Some of your projects will fail. When they do, do not give up. That failure is your teacher. Every engineer in the world has built things that failed. That is how we learn.

This course is organised into seven phases. Each phase teaches you something new, but builds directly on what you learned before. By the end of this course, you will not just know how to programme an Arduino or build a robot. You will understand HOW ENGINEERS THINK. You will see the patterns they use. You will be able to solve problems step-by-step. And you will create things that work in the real world.

Throughout this manual, I speak to you as your teacher. When I say "you should do this", I mean it. When I give you a challenge to solve, it is not optional, it is where the real learning happens. When I warn you about a common mistake, it is because a student before you made that mistake and spent hours trying to fix it. Learn from those lessons without repeating them.

Visit our website at www.muwatta.com.ng. You will find videos showing every single project. You will see working robots in action. You will find a community of other students just like you, learning the same things, building the same projects, and sharing their ideas. Share your work. Learn from others. That is how we all get better.

Most importantly, remember this one thing: You are more capable than you think. The only thing holding you back is you yourself. Let us build something amazing together.

Your teacher,

Mr. Abdullahi Oladipupo Muwatta

Founder and Director, Algorise Tech Explorers

www.muwatta.com.ng

Important Information for Absolute Beginners

You Do Not Need Any Prior Experience

Most students who take this course have never written a line of code before starting. Most have never held an Arduino board before. Most have never studied electricity or electronics. And that is completely perfect. This course teaches you everything you need to know, starting from zero.

What You Actually Need

You need curiosity. You need to be willing to try something, and if it does not work, to try again and figure out why. You need to read instructions carefully. You need to be willing to ask for help when you are confused. And you need patience. That is truly all that is required.

How to Read This Manual

Do not just read the words. Type out every single code example by hand. Do not copy-paste. Do not skip ahead. Read carefully. Do not rush. This course is designed to take 24 weeks, with about 2 hours of class time per week plus time for your own practice. That schedule exists for a reason. Take the time it needs. It is worth it.

What Will Happen When You Get Stuck

You will definitely get stuck. Your code will not work. Your circuit will not light up. Your robot will go the wrong direction. When this happens, do not panic. Do not give up. More than 90% of problems are simple mistakes. A missing semicolon. A wire in the wrong place. A variable name spelled wrong. Use the Debugging Guide in this manual. Ask your teacher. Look carefully at your code. You will find it and fix it.

The Structure of This Course

This course has 7 phases. Phase 1 teaches you to think like an engineer, using only code and paper. Phase 2 teaches you functions and arrays (ways to organise code). Phase 3 teaches you sensors and motors (how machines sense and act). Phase 4 teaches you to build a real moving robot. Phase 5 teaches you specialised robots that do different things. Phase 6 connects your robots to the internet. Phase 7 is your chance to design your own project from scratch.

C++ PROGRAMMING

FOR EMBEDDED SYSTEMS & ROBOTICS

A Secondary School Teaching Manual & Student Workbook

CODE — ELECTRONICS — MICROCONTROLLER — ROBOT — REAL-WORLD PROBLEM

A 24-Week / 96-Hour Embedded Systems & Robotics Programme

Levels 1-10 • 8 Flagship Projects • Arduino & ESP32 • Capstone Engineering Challenge

Algorise Tech Explorers (ATE)

Technology Education Initiative

Table of Contents

Course Roadmap

This manual is designed to be delivered over 24 weeks, at 2 classes per week and 2 hours per class (4 hours/week, 96 hours total). The programme is organised into 7 phases. Programming, electronics, and practical project work are woven together throughout hardware is never separated from the C++ concepts that drive it.

1-4

Programming & Engineering Foundations

Thinking like a programmer, C++ fundamentals, conditions, loops

16h

5-8

Functions, Arrays & Arduino

Functions, arrays, Arduino I/O, first hardware project, analog sensing

16h

9-12

Sensors, Actuators & Automation

Sensors, actuators, PWM, Irrigation project, Poultry Feeder project

16h

13-16

Robotics Fundamentals

2WD chassis build, movement functions, RC robot, obstacle avoidance

16h

17-20

Advanced Robotics Projects

Autonomous navigation, Mopping robot, Soccer robot, Robotics Challenge

16h

21-22

ESP32 & Smart Systems

ESP32 fundamentals, Smart Home/Security, ESP32 IoT Smart Farm Monitor

8h

23-24

Capstone Engineering Project

Team-designed original embedded/robotics system, testing & demo

8h



Central Mental Model

Every project in this course no matter how different it looks on the outside is built from the same three-step engineering pattern: SENSE – THINK – ACT. A sensor senses the world, C++ code thinks (makes a decision), and an actuator acts on that decision. Once a student truly understands this pattern, they can design almost any embedded system.

How to Use This Manual

For s

Each Level and Project follows the same teaching pattern: Real-life problem → Think about the solution → Programming concept → C++ code → Hardware → Test → Debug → Project.

Every module ends with a Guide box: suggested time, equipment, preparation checklist, demonstration procedure, questions to ask, expected mistakes, and assessment criteria.

Do not rush to hardware. A student who cannot reason about an if-statement on paper will not be able to debug it on a robot. Programming and electronics are taught together, but concepts are always understood before they are wired.

Every major project is built in stages (Version 1 → 2 → 3 → Final). Never hand students a finished, working program before they have struggled with a simpler version themselves.

This manual assumes one Arduino Uno (or compatible) + basic sensor/actuator kit per student or pair, and shared robotics/ESP32 kits per group for later phases. A full reuse plan is in the Hardware Reference section so schools do not need to buy new components for every project.

For Students

You do not need any prior programming experience. You do need curiosity and patience engineering involves a lot of testing and fixing.

Type every code example yourself. Do not copy-paste. Typing code forces your brain to notice its structure.

Every 'Challenge' box is compulsory thinking time, not optional extra work. This is where you move from following instructions to designing your own solutions.

When something 'doesn't work,' your first job is not to panic it is to ask: What exactly is not working? The Debugging Guide will train you to answer that question systematically.

Course Philosophy

By the end of this course, a student should not simply say "I learned C++." A student who has completed this manual should be able to say:

"I can use C++ to make a machine sense its environment, make a decision, and perform an action in the real world."

Every lesson, every sensor, every project in this manual exists to build that one capability: SENSE → THINK → ACT.

PHASE 1 Programming & Engineering Foundations (Weeks 1-4, 16 hours)

Before you build anything physical, you need to learn how to THINK like a programmer. This phase teaches you the patterns of thinking that engineers use.

This phase teaches you how to THINK like an engineer before you touch a single wire. before they touch a single wire. By the end of Phase 1, students can break a real-world problem into steps, draw a flowchart, and write a small, correct C++ console program that makes decisions and repeats actions.

LEVEL 1 Thinking Like a Programmer (Week 1)

Learning Objectives

- Understand the difference between hardware, software, and algorithms in embedded systems
- Draw and interpret simple flowcharts that represent real-world processes
- Recognise the Input-Processing-Output pattern in everyday devices
- Write and execute your first C++ program with confidence

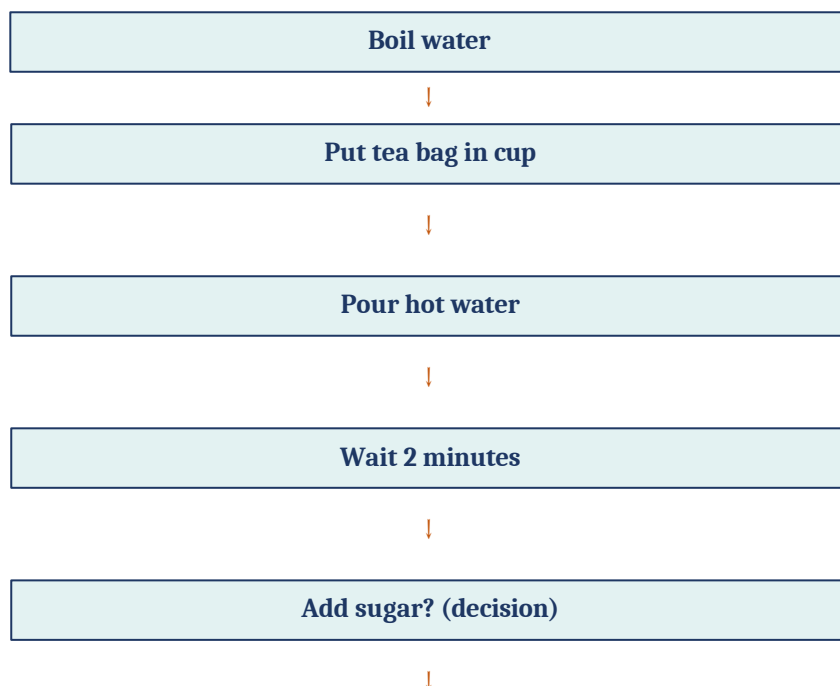
Real-life problem

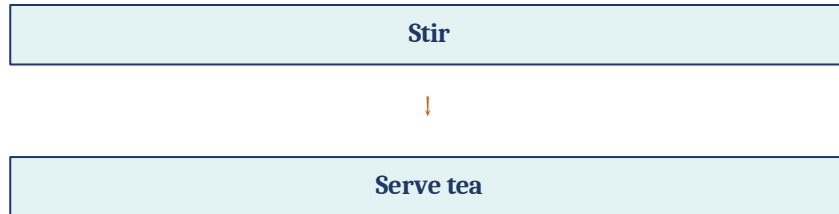
Every day you follow instructions without thinking of them as "code": making tea, crossing the road, using an ATM, waiting for the school bell. A computer needs instructions to be far more precise than a human does.

Think about the solution

Take "making a cup of tea." Ask: what is the very first thing that must happen? What must happen before the water can be poured? What decision might change the steps (sugar or no sugar)? This is exactly what a programmer does before writing any code.

Algorithm: Making Tea





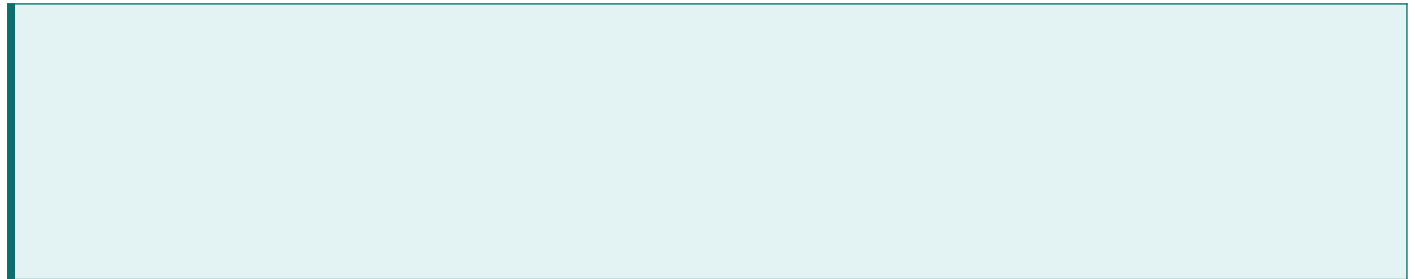
Programming concept

Think of programming this way. Programming means you are giving the computer a set of very precise, ordered instructions. The computer follows these instructions exactly. It does not understand your intention, only your exact words. a computer a precise, ordered set of instructions to solve a problem.

Algorithm = the logical sequence of steps to solve a problem, independent of any programming language.

Flowchart = a diagram that shows an algorithm using shapes: oval (start/end), rectangle (action), diamond (decision), arrows (flow).

Input – Processing – Output (IPO): almost every program and every embedded system read something in, does something with it, and produces a result.



Input – Processing – Output examples

ATM: Input = card + PIN. Processing = check balance. Output = cash + receipt.

Automatic door: Input = motion sensor signal. Processing = "is someone there?". Output = motor opens door.

Washing machine: Input = selected programme + water level. Processing = timing and cycle logic. Output = drum motion, water valve.

This is the same pattern every embedded system in this course will use: Sensor (Input) – C++ code (Processing) – Actuator (Output).

Software vs. Hardware vs. Microcontroller

Hardware

The physical parts you can touch: the board, wires, sensors, motors.

Software

The instructions (code) that tell the hardware what to do.

Microcontroller

A tiny computer on a single chip (CPU + memory + input/output pins) that runs one program to control a physical device e.g. the chip at the heart of an Arduino.

Arduino

A small, beginner-friendly circuit board built around a microcontroller, with pins you can easily connect sensors and actuators to, programmed using the Arduino IDE.

Introduction to C++

C++ is a general-purpose programming language. It is one of the most widely used languages in the world for software that must be fast and close to the hardware operating systems, games, and embedded systems. Arduino "sketches" (the programs you write for Arduino) are written in C++. Every rule you learn about C++ in this manual applies directly to Arduino code there is no separate "Arduino language."

Your first C++ program



```
#include <iostream> using namespace std;

int main() {
    cout << "Hello, Engineer!" << endl; return 0;
}
```

What does each line do?

Learning Objectives

- Declare variables and choose appropriate data types for different sensor values
- Understand why data type selection directly impacts embedded system behavior

- Perform arithmetic, comparison, and logical operations correctly
- Recognise how data types affect memory usage and program correctness

```
#include <iostream> brings in the tools needed to print text to the screen.  
int main() { } every C++ program starts running from main(). Everything between { and } is the body of the program.  
cout << "..." << endl; cout ("character output") prints text; endl moves to a new line.  
return 0; tells the operating system the program finished successfully.
```



STUDENT CHALLENGE

Change the message so it prints your own name. Add a second cout line that prints your class/grade.

What happens if you forget the semicolon at the end of a line? Try it and read the error message.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Projector; any C++ environment (Replit, online compiler, or installed IDE) no hardware needed yet.

Preparation checklist: Have 4-5 relatable everyday processes ready (tea, traffic light, school bell, ATM, washing machine). Prepare blank flowchart templates.

Demonstration procedure: Live-draw a flowchart for the school bell system on the board, then walk it into IPO form before showing any code.

Questions to ask students: What is the input in a washing machine? What decision does an ATM make? What is the difference between an algorithm and a flowchart?

Expected mistakes: Students confuse 'hardware' and 'software'; students try to write code before they can describe the steps in plain English.

Assessment criteria: Student can draw a correct flowchart for a new everyday process and correctly label its Input, Processing and Output.

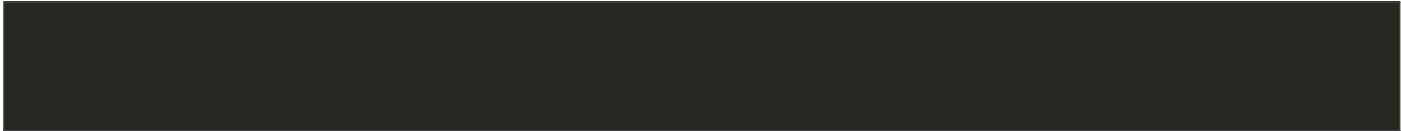
LEVEL 2 C++ Fundamentals (Week 2)

Real-life problem

A weather station needs to remember the current temperature. A robot needs to remember how far away an obstacle is. A program needs a place to store values that can change this is what a variable is for.

Variables

A variable is like a labelled box in your computer memory. You write a label on the box (like "temperature" or "distance"). Inside the box, you store a number or value that can change. When you need that value later, you just use the label. in the computer's memory that holds a value. Think of it as a labelled container: the label (name) never changes, but what's inside (the value) can.



```
int age = 14;  
float temperature = 28.5; bool lightOn = true;
```

Every variable has a type (what kind of value it can hold), a name (chosen by the programmer), and a value.

Data types students actually need at this stage

Learning Objectives

- Implement decision-making logic using if, else if, and else statements
- Combine multiple conditions using logical operators (AND, OR, NOT)
- Choose between if-else chains and switch statements appropriately
- Debug conditional logic systematically when decisions do not execute as intended

int
Whole numbers
int score = 87;
Counting, indexes, digital pin states
float
Decimal numbers

float temp = 28.5;

Sensor readings are rarely whole numbers

double

Decimal numbers, more precision

double pi = 3.14159265;

When more decimal accuracy is needed

char

A single character

char grade = 'A';

Single letters, menu choices

bool

true or false

bool doorOpen = false;

On/off, yes/no states the basis of decisions

Why does this matter for embedded systems?

A digital pin can only be HIGH or LOW a bool. A temperature sensor gives a decimal reading a float. Choosing the right data type is not academic; it directly affects how much memory your microcontroller uses and whether your logic is correct.

Operators

Arithmetic

+ - * / %

totalPrice = quantity * unitPrice;

Comparison

== != > < >= <=

if (temperature > 30) is it hot?

Logical

&& || !

if (isDark && motionDetected) turn on light only if both are true

Assignment

= += -= *= /=

score += 10; means score = score + 10;

Input and output

Console I/O will not be the destination of this course (hardware is), but it is the fastest way to practise variables and logic before wiring anything.

```
#include <iostream> using namespace std;

int main() {
    int age;
    cout << "Enter your age: "; cin >> age;
    cout << "Next year you will be " << (age + 1) << endl; return 0;
}
```

Mini Project *Student Score/Grade Calculator*

Build a program that asks for three subject scores, calculates the average, and prints it. (Grading logic with Think about your day. If it rains, you take an umbrella. Else, you do not. This is exactly what if/else does. The computer checks a condition. If it is true, the computer does one thing. If it is false, the computer does something else. added next week for now, focus on variables, arithmetic, and output.)

STUDENT CHALLENGE

Store your name in a char array or string and print a full sentence with your name and average score. Use % to calculate what percentage 45 out of 60 marks represents.

Predict what int result = 7 / 2; stores before running it. Were you right? Why?

GUIDE

Suggested time: 4 hours (2 classes in a week) Equipment needed: Any C++ compiler/IDE.

Preparation checklist: Prepare 5-6 short 'predict the output' snippets for warm-up.

Demonstration procedure: Live-code the score calculator, deliberately using the wrong data type once (e.g. int for an average) so students see the truncation bug.

Questions to ask students: Why can't a bool store the number 5? What is the difference between = and ==? What does 7 / 2 give in int arithmetic, and why?

Expected mistakes: Confusing = (assignment) with == (comparison); expecting integer division to give a decimal result; forgetting to declare a variable's type.

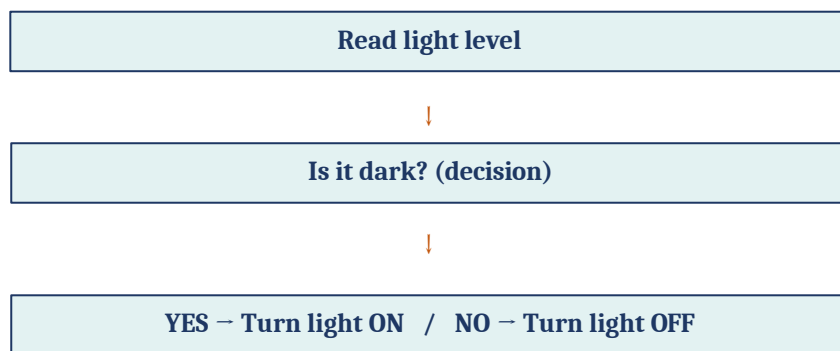
Assessment criteria: Score/grade calculator runs correctly and uses appropriate data types for each value.

LEVEL 3 Decision Making (Week 3)

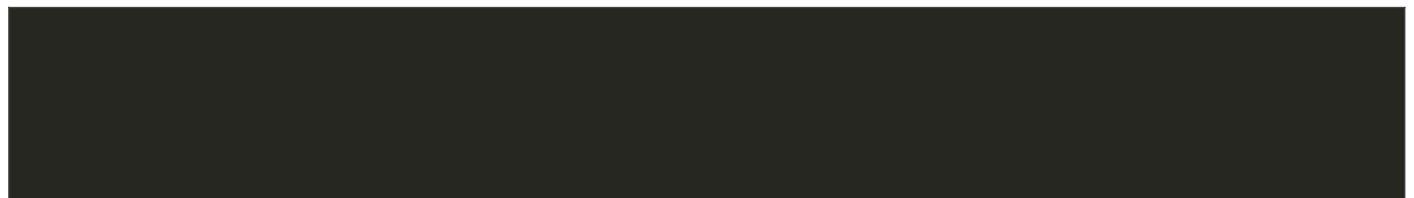
Real-life problem

Imagine an automatic street light. When it becomes dark, the light should turn ON. When it becomes bright, it should turn OFF. A program needs a way to choose between different actions based on a condition.

Think about the solution



Programming concept: if / else



```
if (lightLevel < threshold) {  
  // turn light ON  
} else {  
  // turn light OFF  
}
```

This is the exact logic we will later run on real hardware using an Arduino and an LDR (light sensor) in Level 8. Right now we practise the decision-making pattern itself.

Learning Objectives

- Choose the correct loop structure (for, while, do-while) for each scenario
- Write loop conditions that terminate correctly and avoid infinite loops
- Manually trace loop execution to predict program behavior before running code
- Apply loops to repeated sensor reading and motor control operations

if / else if / nested if / switch

```
int temperature = 38;

if (temperature > 40) {
    cout << "DANGER: Overheating!" << endl;
} else if (temperature > 30) {
    cout << "Warning: Getting hot" << endl;
} else {
    cout << "Normal temperature" << endl;
}
```

A switch statement is useful when one variable can take several exact values for example, a simple traffic-light state:

```
int lightState = 2; // 1 = Red, 2 = Yellow, 3 = Green switch (lightState) {
```

```
case 1:
    cout << "STOP" << endl; break;
case 2:
    cout << "GET READY" << endl; break;
case 3:
    cout << "GO" << endl; break;
default:
    cout << "Invalid state" << endl;
}
```

Logical operators in decisions

A security alarm example: sound the alarm only if motion is detected AND the system is armed.

```
bool motionDetected = true; bool systemArmed = true;

if (motionDetected && systemArmed) {
    cout << "ALARM! Intruder detected." << endl;
}
```

Practical decision-making scenarios to code this week

Automatic fan: if temperature > 30, fan ON, else fan OFF.

Water-level controller: if level < minimum, pump ON; if level >= maximum, pump OFF.

Smart street light with nested condition: if it is night AND motion is detected, full brightness; if night but no motion, dim brightness; if day, OFF.

Temperature warning system using else if chains for Normal / Warning / Danger bands.

Mini Project Automatic Traffic-Light Decision System

Write a console program that stores the current second of a repeating 9-second cycle (0-2 Red, 3-5 Yellow, 6-8 Green) and prints the correct light using if/else if. This is deliberately similar to the real traffic-light sequence students will build with LEDs in Level 4.

STUDENT CHALLENGE

Add a fourth condition for a pedestrian-crossing button that forces Red immediately. Rewrite your traffic-light program using switch instead of if/else if.

What happens if two if conditions could both be true at once? Design a nested-if street light that also considers rain (dimmer in rain).

GUIDE

Suggested time: 4 hours (2 classes in a week) Equipment needed: Any C++ compiler/IDE.

Preparation checklist: Prepare a printed flowchart of the street light logic for students to code from directly.

Demonstration procedure: Live-build the temperature warning system step by step: start with only if, add else, then add else if, showing how the logic tightens each time.

Questions to ask students: What is the difference between else if and a separate if? Why does order matter in a chain of else if statements? When is switch better than if/else if?

Expected mistakes: Using = instead of == inside a condition; writing overlapping else if ranges that never get reached; forgetting break in a switch and causing 'fall-through.'

Assessment criteria: Programming challenge: student is given a new real-world scenario (e.g. lift/elevator door logic) and must design the condition structure independently.

LEVEL 4 Loops (Week 4)

Real-life problem

A robot scanning for obstacles doesn't check distance once it checks again and again. A blinking LED doesn't turn on once it must turn on, off, on, off, forever. Repetition needs a loop, not fifty copies of the same instruction.

Learning Objectives

- Design functions that solve a single, clear problem
- Use parameters to make functions flexible and reusable
- Return values from functions and use them in subsequent calculations
- Organise complex programs into well-named functions for readability

Programming concept: for, while, do-while

```
for (int i = 1; i <= 5; i++) { cout << "Blink " << i << endl;
}
```

for loop: best when you know how many times to repeat (a counter).

Learning Objectives

- Store multiple related values in arrays instead of separate variables
- Iterate through arrays using loops to process sensor data efficiently
- Calculate aggregate values (average, minimum, maximum) from array data
- Apply array processing to real sensor readings from weather stations and irrigation systems

while loop: best when you repeat until a condition becomes false, and you don't know in advance how many times that will take (e.g. 'while distance is greater than 10').

do-while loop: like while, but always runs at least once useful for menus that must show at least one time before checking whether to repeat.

```
int distance = 100; while (distance > 10) {
cout << "Moving forward, distance = " << distance << endl; distance = distance - 10; //
simulate getting closer
}
cout << "Stopping - obstacle close!" << endl;
```

Connecting loops to hardware (preview)

In Level 7 you will meet the real Arduino loop() function, which the microcontroller repeats automatically forever this is exactly the while(true) idea. Every Arduino sketch you ever write is really one giant infinite loop.

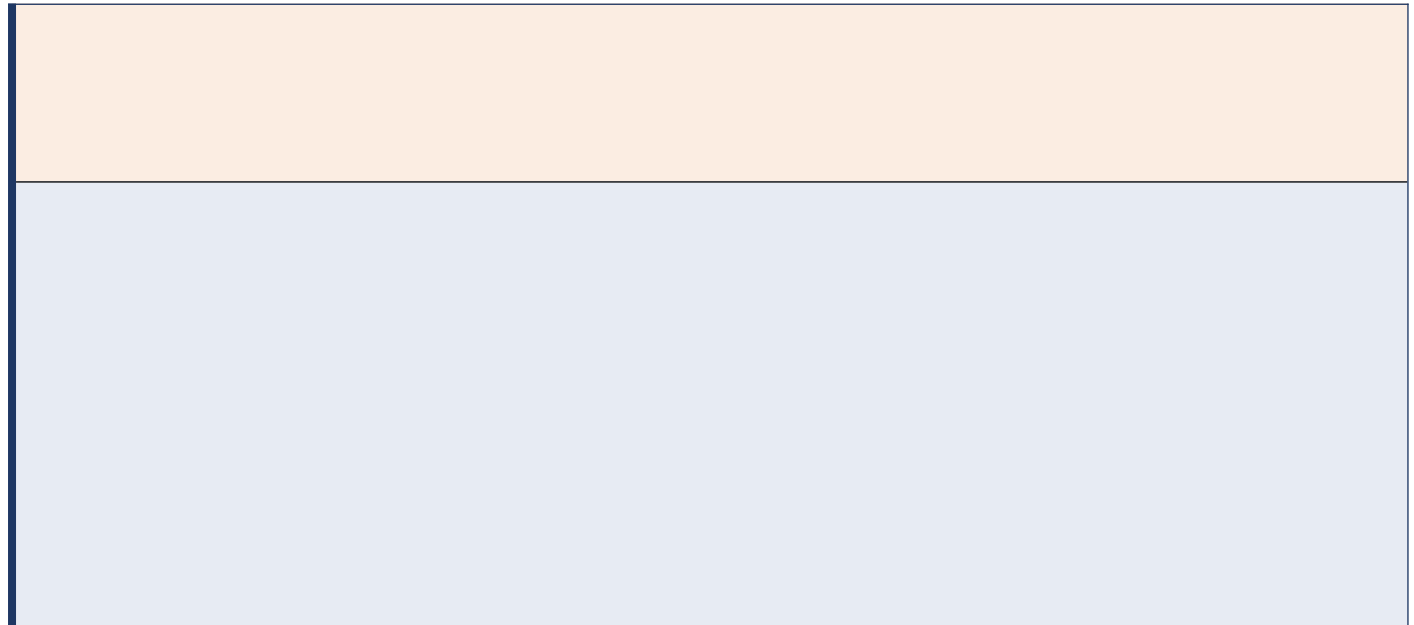
```
// this is a PREVIEW - real Arduino code, explained fully in Level 7 void loop() {
digitalWrite(LED_PIN, HIGH); delay(500); digitalWrite(LED_PIN, LOW); delay(500);
}
```

Mini Project Countdown, LED Simulation & Simple Menu

Countdown timer: use a for loop to count down from 10 to 0 with a message at 0.

LED blink simulation: use a while loop to print 'LED ON' / 'LED OFF' alternately ten times, simulating what will later be a real blinking LED.

Simple menu system: use a do-while loop to repeatedly show a menu (1: Add score, 2: Show average, 3: Exit) until the user chooses Exit.



STUDENT CHALLENGE

Modify the countdown to count down in steps of 2 instead of 1. Rewrite the LED blink simulation using a for loop instead of while.

Combine a loop with an if/else you learned last week: print 'Even' or 'Odd' for numbers 1 to 20.

GUIDE

Suggested time: 4 hours (2 classes in a week) + Assessment Equipment needed: Any C++ compiler/IDE.

Preparation checklist: Prepare a short 'programming challenge' assessment covering variables, decisions and loops together.

Demonstration procedure: Trace a for loop on the whiteboard variable-by-variable (i = 1, 2, 3...) before running it, so students see the loop as a sequence of steps, not magic.

Questions to ask students: What happens if you forget i++ inside a for loop? What is the difference between while and do- while in terms of how many times the body can run?

Expected mistakes: Infinite loops from a condition that never becomes false; off-by-one errors (looping one time too many or too few); confusing loop counters with the values being processed.

Assessment criteria: End-of-phase programming challenge: given a new small problem, student correctly chooses between if/else, for, and while, and produces a working, readable program.

PHASE 2 Functions, Arrays & Arduino (Weeks 5-8, 16 hours)

You now know how to think. This phase teaches you to organise that thinking into reusable pieces (functions) and to manage multiple pieces of data (arrays).

Students now have enough C++ to write real logic. This phase teaches them to organise that logic (functions), store multiple pieces of data (arrays), and for the first time run their code on real hardware.

LEVEL 5 Functions (Week 5)

Real-life problem

Imagine writing the same 5 lines of motor-control code every time a robot needs to move forward. If you need to change how 'forward' works, you would have to find and fix it in ten different places. Functions solve this: write the logic once, name it, and call it wherever needed.

Programming concept

A function is like a recipe. You write the recipe once. You give it a name. Then whenever you need that recipe, you just call its name instead of writing all the steps again. Functions make your code much shorter and easier to read. a named, reusable block of code that performs one clear task.

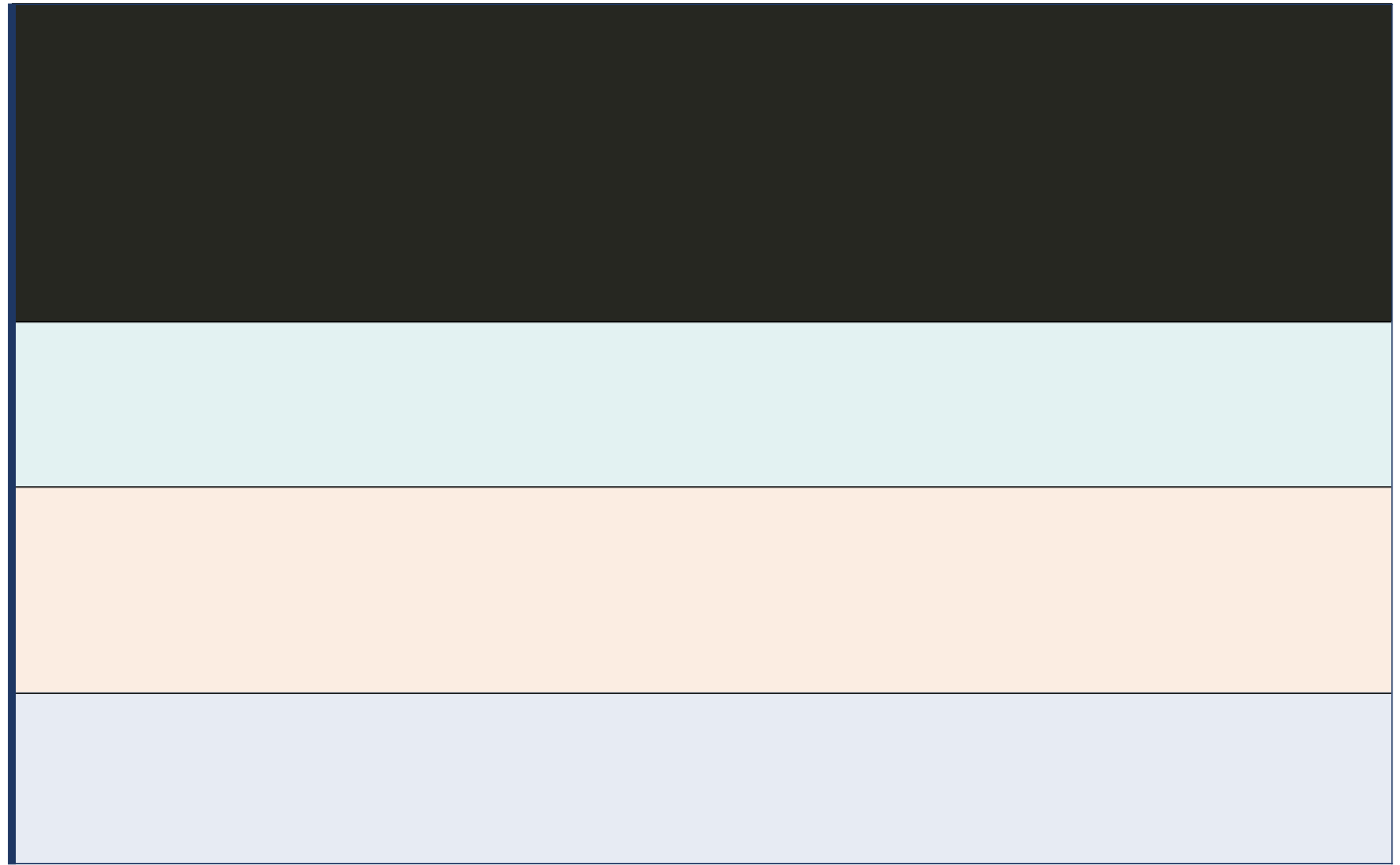
Parameters let you pass information into a function so it can behave differently each time.

A return value lets a function send a result back to whoever called it.



```
void moveForward() {  
    // motor control instructions go here cout << "Robot moving forward" << endl;  
}  
  
void stopRobot() {  
    cout << "Robot stopped" << endl;  
}  
  
int main() {  
    moveForward(); stopRobot(); return 0;  
}
```

Functions with parameters and return values



```
int addScores(int a, int b) { return a + b;
}

int main() {
int total = addScores(15, 20);
cout << "Total: " << total << endl; return 0;
}
```

Why did we write it this way?

void means the function does not send back a value it just performs an action (like moving a motor). A function that calculates something (like addScores) uses a type such as int to say what kind of value it returns. Robotics code becomes readable once movements are functions: moveForward(); turnLeft(); stopRobot(); reads almost like English.

STUDENT CHALLENGE

Write a function isEven(int n) that returns true or false.

Write turnLeft() and turnRight() functions (as console messages for now) and call them in a sequence that simulates a robot navigating a square path.

Give moveForward() a parameter called speed and print a message that includes it.

GUIDE

Suggested time: 4 hours (2 classes in a week) Equipment needed: Any C++ compiler/IDE.

Preparation checklist: Prepare a set of 'robot movement' function stubs (moveForward, moveBackward, turnLeft, turnRight,

stopRobot) students will reuse and expand for the rest of the course.

Demonstration procedure: Show the same program written (a) with all logic dumped into main(), then (b) reorganised into functions let students see the readability difference directly.

Questions to ask students: What is the difference between a function that returns void and one that returns int? What is a parameter, and how is it different from a return value?

Expected mistakes: Forgetting return in a non-void function; mismatching parameter types when calling a function; declaring a function after main() without a prototype and getting a compile error.

Assessment criteria: Student can design and correctly call at least 4 custom functions, including one with a parameter and one with a return value.

LEVEL 6 Arrays and Basic Data Handling (Week 6)

Real-life problem

A weather station doesn't take one temperature reading it takes many over time, and often needs to find the average, the highest, or the lowest. Storing each reading in a separate variable (temp1, temp2, temp3...) does not scale. An array stores many values of the same type under one name.

Programming concept: Arrays

```
int sensorReadings[5] = {23, 45, 67, 12, 89};

for (int i = 0; i < 5; i++) {
    cout << "Reading " << i << ": " << sensorReadings[i] << endl;
}
```

Array indexes start at 0, not 1 sensorReadings[0] is the first reading.

The array's size must be known (or fixed) this matches real embedded systems, which have limited memory and cannot grow arrays freely.

Loops and arrays are natural partners: a for loop is the normal way to process every element.

Processing multiple sensor values

```
int readings[5] = {23, 45, 67, 12, 89}; int total = 0;
int highest = readings[0];

for (int i = 0; i < 5; i++) { total += readings[i];
if (readings[i] > highest) { highest = readings[i];
}
}

float average = (float)total / 5;
cout << "Average: " << average << ", Highest: " << highest << endl;
```

Strings

In plain C++ a string of characters can be stored as a char array (char name[] = "Musa";) or, more conveniently, using the string type from the <string> library. On Arduino, the String class is commonly used for short pieces of text such as sensor labels or Bluetooth commands this will be used from Level 7 onward.

STUDENT CHALLENGE

Store 7 days of simulated temperature readings in an array and print the day with the lowest temperature.

Write a function that takes an array and its size as parameters and returns the average this is exactly how you will later average multiple ultrasonic sensor readings to reduce noise.

What happens if you try to access readings[10] on a 5-element array? Try it and explain what you observe.

GUIDE

Suggested time: 4 hours (2 classes)

Preparation checklist: Prepare a small dataset (e.g. a week of temperature/rainfall figures) for the practical exercise.
 Demonstration procedure: Live-code the average/highest/lowest pattern step by step, pausing to trace the loop and the array indices on the board.

Questions to ask students: Why does the first element have index 0? What real embedded scenario would need to store multiple sensor readings?

Expected mistakes: Off-by-one errors when looping through arrays (using `<=` instead of `<` against the size); forgetting that array indexing starts at 0; integer division errors when computing averages.

Assessment criteria: Student can store, loop through, and correctly summarise (average/min/max) a set of simulated sensor values.

LEVEL 7 Arduino Fundamentals (Week 7)

Arduino sketches ARE C++

Every function, loop, if-statement, variable and data type you have learned so far works exactly the same way on Arduino. An Arduino 'sketch' is a C++ program with two required functions: `setup()` (runs once) and `loop()` (repeats forever). There is no separate 'Arduino language' to learn.

Arduino board anatomy

Microcontroller chip the 'brain' that runs your compiled program.

Analog input pins (A0-A5) read a range of voltage values, not just on/off (used for sensors like potentiometers and LDRs).

PWM pins (marked with ~) can simulate an analog output using rapid on/off pulses (used for motor speed, LED brightness).

GND pins ground, the reference '0V' that must be shared by every component in the circuit.

USB port power and programming connection to a computer.

[illegible][illegible]

--	--

`pinMode(pin, MODE)`

Configures a pin as INPUT or OUTPUT before use.

`digitalWrite(pin, HIGH/LOW)`

Sets an output pin fully on (HIGH, ~5V) or off (LOW, 0V).

`digitalRead(pin)`

Reads whether an input pin is HIGH or LOW.

`analogRead(pin)`

Reads a value from 0-1023 representing voltage on an analog pin.

`analogWrite(pin, value)`

Outputs a PWM signal (0-255) used for brightness/speed control.

`delay(ms)`

Pauses the program for the given number of milliseconds.

`Serial.begin(baud) / Serial.println()`

Opens/uses the Serial Monitor to send text from the board to your computer essential for debugging.

Anatomy of a sketch



```
void setup() {  
  // runs ONCE when the board powers on or resets  
}  
  
void loop() {  
  // runs FOREVER, over and over, after setup()  
}
```

PROJECT 1 LED Control

Problem

How do we make the Arduino turn a real, physical light on and off under program control? This is the smallest possible bridge between code and the physical world.

Learning objectives

- Understand `pinMode()`, `digitalWrite()`, and `delay()` completely.
- Wire a simple output circuit correctly and safely (including why a resistor is required).
- Practise the Version 1 → Final Version development process.

Components (Bill of Materials)

Arduino Uno (or compatible)

1

Main controller

LED (any colour)

1

Visible output device

220Ω resistor

1

Limits current so the LED is not damaged

Breadboard

1

Solderless circuit assembly

Jumper wires

2-3

Electrical connections

USB cable

1

Power and programming

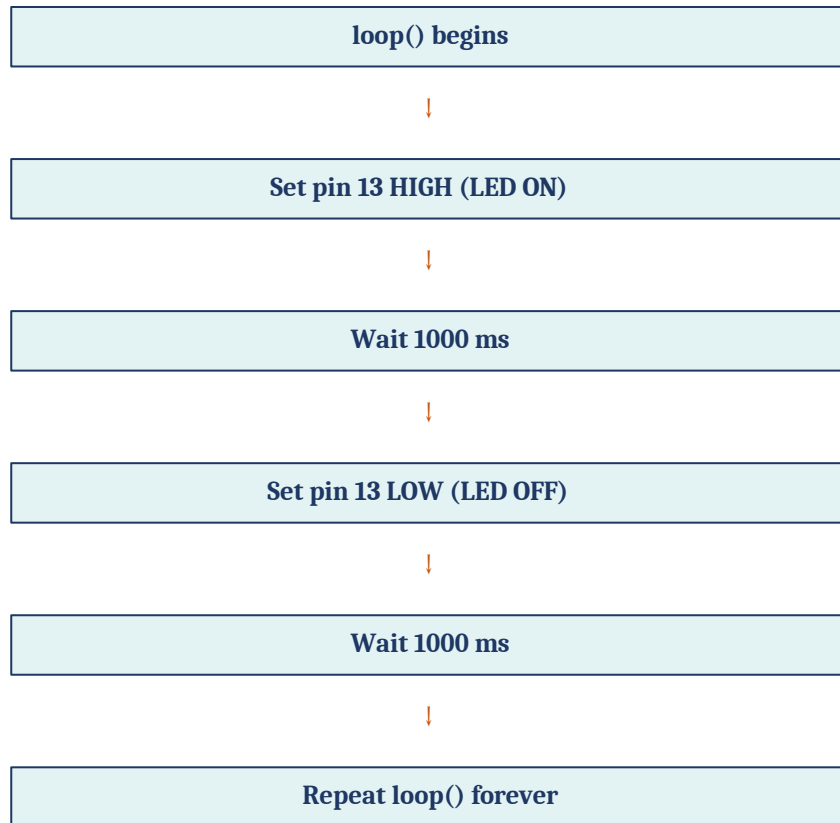
Circuit

LED anode (long leg) → 220Ω resistor → Arduino digital pin 13. LED cathode (short leg) → Arduino GND. The resistor can sit on either leg, but must always be present in series with the LED.

How the system works

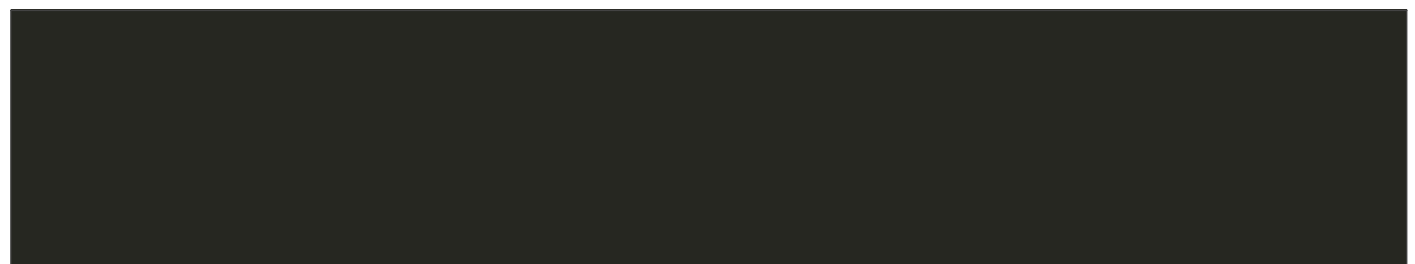
setup(): configure pin 13 as OUTPUT





Build in stages

Version 1 Smallest working program



```
void setup() { pinMode(13, OUTPUT);  
}
```

```
void loop() {  
  digitalWrite(13, HIGH);  
}
```

Test it: the LED should turn on and stay on. This confirms the wiring and pin number are correct before adding any timing logic.

Version 2 Add blinking

```
void setup() {
```

```
pinMode(13, OUTPUT);  
  
}  
  
void loop() {  
  digitalWrite(13, HIGH); delay(1000); digitalWrite(13, LOW); delay(1000);  
}
```

Final Version Named pin, adjustable timing

```
const int LED_PIN = 13;  
const int BLINK_TIME = 500; // milliseconds  
  
void setup() { pinMode(LED_PIN, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED_PIN, HIGH); delay(BLINK_TIME); digitalWrite(LED_PIN, LOW);  
  delay(BLINK_TIME);  
}
```

Why did we write it this way?

Naming the pin (LED_PIN) and the timing (BLINK_TIME) as constants means changing the whole program's behaviour requires editing one line, not hunting through the code. This habit becomes essential once programs grow to control multiple motors and sensors.

What happens if we change this value?

Change BLINK_TIME to 50 the blink becomes almost invisible flicker.

Change LED_PIN to a pin with nothing connected nothing visible happens, but no error occurs either (a good lesson: no error message ≠ correct behaviour).

Remove the resistor conceptually ask students why this would be dangerous for the LED (excess current).

Common errors

LED does not light at all

LED inserted backwards, wrong pin number in code, loose jumper wire

Upload fails / port not found

Wrong board or COM port selected in IDE, USB cable is 'charge-only'

LED stays on constantly

Missing delay() calls, or digitalWrite(LOW) never reached

STUDENT CHALLENGE

Make the LED blink twice quickly, then pause for 2 seconds, repeating this pattern (a 'distress signal' rhythm). Add a second LED on a different pin and alternate them (when one is ON, the other is OFF).

Rewrite the blink using a for loop that blinks exactly 10 times and then stops.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: 1 Arduino Uno + LED + resistor + breadboard + jumpers per student/pair; Arduino IDE installed on all machines beforehand.

Preparation checklist: Install Arduino IDE and test board drivers on every classroom computer before class. Pre-cut jumper wires if possible to save time.

Demonstration procedure: Wire and upload Version 1 live, then deliberately unplug the LED to show that 'no error' does not mean 'correct behaviour' reinforces the debugging mindset early.

Questions to ask students: What does pinMode() need to know before digitalWrite() can be used? Why do we need a resistor?

--	--

What is the difference between HIGH and LOW?

Expected mistakes: Selecting the wrong board/port in the IDE; inserting the LED backwards; forgetting pinMode() and getting inconsistent behaviour.

Assessment criteria: Student's LED blinks at a specified custom rate, and student can explain every line of their final code aloud.

LEVEL 8 (Introduction) Analog Sensing (Week 8)

Real-life problem

A street light should not simply be ON or OFF based on a button it should respond to how bright it actually is outside. This needs a sensor that reports a range of values, not just HIGH/LOW.

Programming concept: analogRead() and thresholds

analogRead() returns a number from 0 (0V) to 1023 (5V) representing the voltage on an analog pin. Combined with the if/else logic from Level 3, this lets a program compare a real-world brightness level against a threshold and decide what to do the exact same pattern as the street light example from Level 3, now running on real hardware.

Potentiometer the simplest analog sensor

A potentiometer is a variable resistor: turning its knob changes the voltage it outputs. It is the easiest way to practise analogRead() before using a real environmental sensor.

```
const int POT_PIN = A0;

void setup() { Serial.begin(9600);
}

void loop() {
  int value = analogRead(POT_PIN); Serial.println(value); delay(200);
}
```

Introducing the LDR (Light Dependent Resistor)

An LDR's resistance drops as light increases. Wired in a voltage-divider with a fixed resistor, it produces a voltage that analogRead() can measure, giving a number that rises or falls with brightness.

PROJECT Automatic Night Light

[illegible]

Arduino Uno

1

Main controller

LDR (light sensor)

1

Detects ambient brightness

10k Ω resistor

1

Voltage-divider partner for the LDR

LED

1

Output light

220Ω resistor

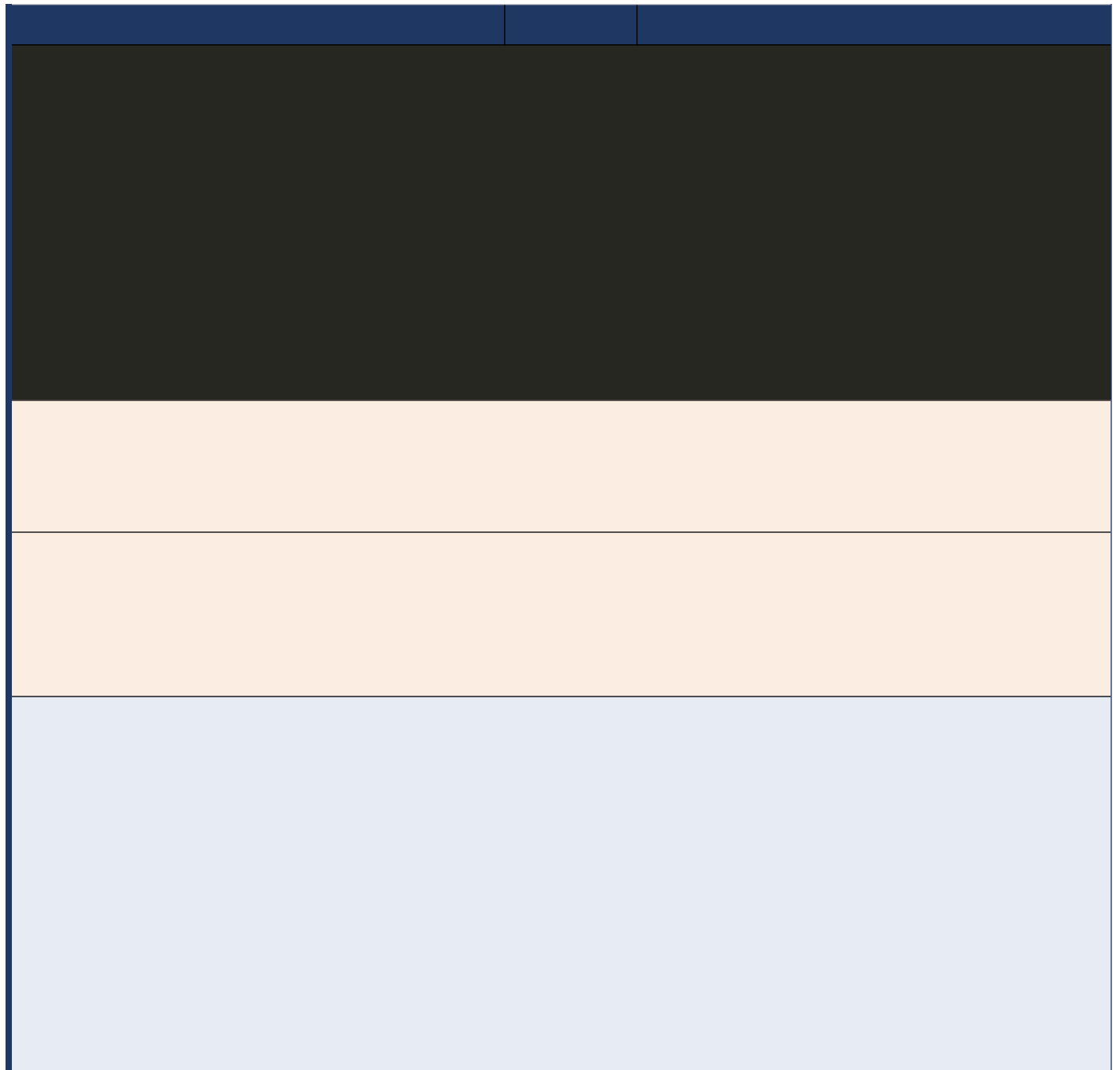
1

Protects the LED

Breadboard + jumper wires

1 set

Assembly



```
}

void loop() {
  int lightLevel = analogRead(LDR_PIN);
  Serial.println(lightLevel); // watch this to choose your threshold

  if (lightLevel < DARK_THRESHOLD) { digitalWrite(LED_PIN, HIGH); // dark -> light ON
  } else {
    digitalWrite(LED_PIN, LOW); // bright -> light OFF
  }
}
```

```
delay(200);
```

```
}
```

Calibration is an engineering skill, not a bug

There is no universal 'correct' DARK_THRESHOLD value it depends on the exact LDR, resistor, and room lighting. Students must watch the Serial Monitor values in bright and dark conditions and choose a sensible threshold between them. This is the same calibration skill used later for soil moisture and ultrasonic thresholds.

STUDENT CHALLENGE

Add a second LED that turns on only when it's very dark (a stricter, lower threshold) simulating a 'security light' that needs it darker before activating.

Print 'DARK' or 'BRIGHT' to the Serial Monitor instead of just numbers.

Replace the LDR with the potentiometer and manually simulate day/night by turning the knob.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Arduino Uno + LDR + 10k Ω resistor + LED + 220 Ω resistor + breadboard per student/pair. Preparation checklist: Test LDR values in the actual classroom lighting beforehand so you can guide students to a sensible threshold quickly.

Demonstration procedure: Open the Serial Monitor and cover the LDR with your hand live, showing the class the value change in real time before writing any if-statement.

Questions to ask students: What does analogRead() return, and what range of values is possible? Why is there no single correct threshold value for every room?

Expected mistakes: Choosing a threshold without testing (e.g. copying a value from a textbook that doesn't match the room); wiring the LDR without a partner resistor (fixed HIGH or LOW reading with no variation).

Assessment criteria: Night light responds correctly to hand-covering the LDR, and student can explain how they chose their threshold value.

PHASE 3 Sensors, Actuators & Automation (Weeks 9-12, 16 hours)

Your Arduino board needs to understand the world around it. This phase teaches you sensors (how machines see, feel, and measure) and actuators (how machines act on decisions).

This phase gives students the two remaining ingredients of every embedded system: ways to sense the world and ways to act on it, and introduces the central mental model of the course: SENSE $\rightarrow$ THINK $\rightarrow$ ACT. It closes with two complete automation projects.

SENSE → THINK → ACT

A sensor SENSES the environment and produces a reading. C++ code THINKS it compares the reading to a threshold or condition and decides what should happen. An actuator ACTS on that decision by doing something physical: moving, lighting up, pumping, sounding. Every project from this point in the course is this same pattern, repeated with different hardware.

LEVEL 8 Sensors in Depth (Week 9)

Real-life problem

A robot cannot see, feel heat, or smell soil the way a human does. Sensors are the robot's substitute senses each one converts one kind of physical quantity (distance, light, temperature, moisture) into an electrical signal the microcontroller can read with `digitalRead()` or `analogRead()`.

Push Button



What it detects

A simple press/release (digital, mechanical input).

How it works (basic level)

Pressing the button completes or breaks a circuit. Without a defined resting state the pin can 'float' and read randomly, so a pull-up or pull-down resistor (or INPUT_PULLUP mode) is required.

Pins

One digital input pin.

Wiring notes

One leg to the digital pin and GND (using INPUT_PULLUP, pressed = LOW), or with an external pull-down resistor to +5V (pressed = HIGH).

Library required

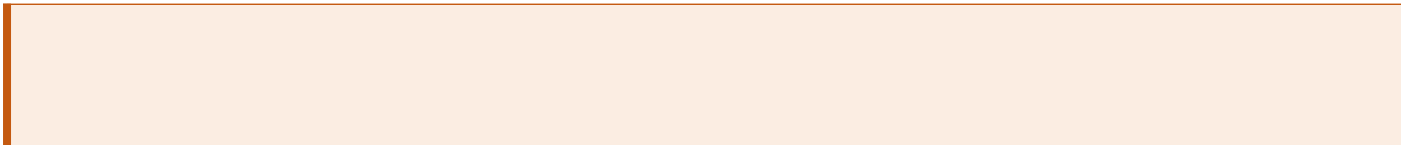
None.

```
const int BUTTON_PIN = 2;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  bool pressed = digitalRead(BUTTON_PIN) == LOW; Serial.println(pressed ? "PRESSED" :
  "released"); delay(100);
}
```

Testing: Open Serial Monitor and press the button; the reading must change cleanly, not flicker randomly when not pressed.



Common errors

Floating input (no pull-up/down) causing random readings.

Confusing 'pressed = LOW' logic when using INPUT_PULLUP a common source of inverted logic bugs.

Real-world applications: Manual override switches, mode selection, start/stop controls on almost every project in this course.

Potentiometer

What it detects

A manually adjustable analog value (0-5V range).

How it works (basic level)

A variable resistor with a wiper; turning the shaft changes the voltage at the middle pin.

Pins

One analog input pin (e.g. A0); outer legs to 5V and GND.

Wiring notes

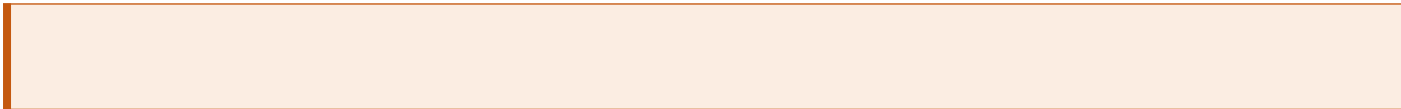
Left leg → GND, right leg → 5V, middle (wiper) → analog pin.

Library required

None.

```
int value = analogRead(A0); // 0 to 1023
```

Testing: Turn the knob fully both ways while watching Serial Monitor; value should sweep smoothly 0–1023.



Common errors

Wiper pin swapped with a supply pin, giving a fixed reading that never changes.

Real-world applications: Manual speed/threshold adjustment, simulating any analog sensor for testing code before the real sensor arrives.

LDR (Light Dependent Resistor)

What it detects

Ambient light level.

How it works (basic level)

Resistance falls as light increases; paired with a fixed resistor in a voltage divider, this produces a voltage analogRead() can measure.

TRIG (digital output), ECHO (digital input), VCC (5V), GND.

Wiring notes

TRIG and ECHO to two separate digital pins; VCC to 5V, GND to GND.

Library required

None required (can be coded directly with pulseIn()); NewPing library simplifies multi-sensor use.

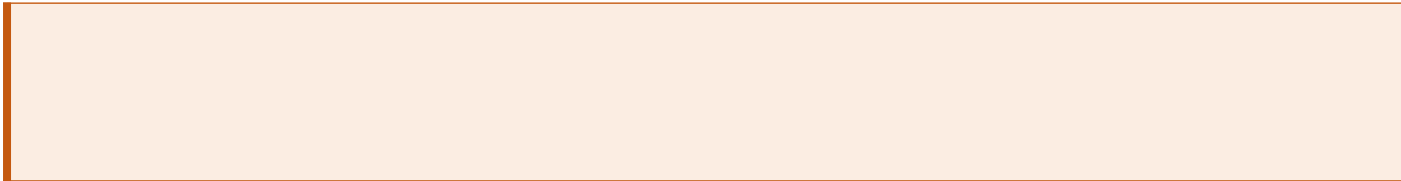
```
const int TRIG_PIN = 9; const int ECHO_PIN = 10;

long readDistanceCM() { digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN, LOW);


```

```
long duration = pulseIn(ECHO_PIN, HIGH); long distanceCM = duration * 0.034 / 2; return
distanceCM;
}
```

Testing: Hold a flat object at a known distance (e.g. 30 cm ruler) and confirm the Serial Monitor reading is close to that value.



Common errors

ECHO pin not level-shifted on 3.3V boards (Uno is fine at 5V).
Reading 0 or very large numbers usually a wiring mix-up between TRIG and ECHO. Angled or soft (sound-absorbing) surfaces give unreliable readings.
Real-world applications: Obstacle-avoiding robots, parking sensors, water-tank level sensing, automatic doors.

IR (Infrared) Sensor

What it detects

Presence of an object (obstacle sensor) or a line's edge (line-following sensor), using reflected infrared light.

How it works (basic level)

An IR LED emits infrared light; a phototransistor detects how much bounces back. Most breakout modules output a simple digital HIGH/LOW already adjusted by an onboard comparator and trimmer.

Pins

OUT (digital, sometimes analog), VCC, GND.

Wiring notes

OUT to a digital input pin; VCC to 5V; GND to GND.

Library required

None.

Testing: Wave a hand in front of the sensor and confirm the digital reading flips.

Common errors

Sensitivity trimmer not adjusted for ambient light/surface colour.

Confusing active-LOW logic (many IR modules output LOW when an object is detected, not HIGH).

Real-world applications: Ball/line detection in the soccer robot, simple obstacle bumpers, counting objects on a line.

DHT11 / DHT22 Temperature & Humidity Sensor

[illegible]

What it detects

Air temperature and relative humidity.

How it works (basic level)

An internal chip measures both quantities and sends them as a timed digital signal on a single data wire.

Pins

DATA (digital, needs a 10kΩ pull-up resistor), VCC, GND.

Wiring notes

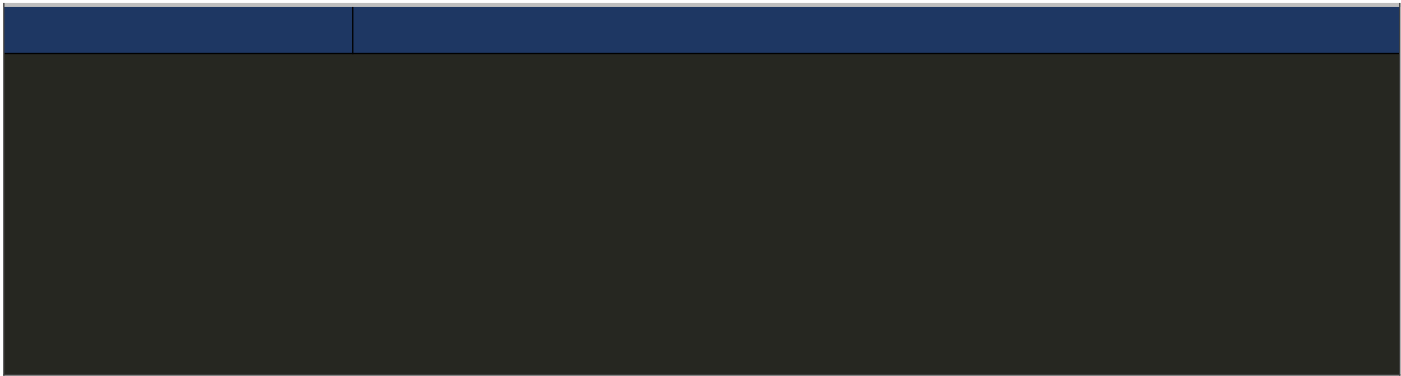
DATA to a digital pin, with a 10kΩ resistor between DATA and 5V; VCC to 5V; GND to GND.

Library required

DHT sensor library (Adafruit).

```
#include <DHT.h> #define DHTPIN 2 #define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
```

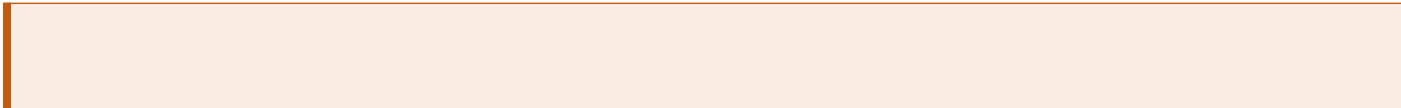
```
void setup() { Serial.begin(9600); dht.begin();
```



```
}

void loop() {
float temp = dht.readTemperature(); float hum = dht.readHumidity();
Serial.print("Temp: "); Serial.print(temp); Serial.print(" C, Humidity: ");
Serial.println(hum); delay(2000);
}
```

Testing: Compare reading against a known thermometer; breathe gently near the sensor and watch humidity rise.



Common errors

Reading NaN usually a missing pull-up resistor or reading too frequently (DHT11 needs ~1-2 s between reads).

Real-world applications: Smart farm monitors, greenhouse control, weather stations, HVAC automation.

Capacitive Soil Moisture Sensor

What it detects

How much moisture is in soil.

How it works (basic level)

Measures the soil's capacitance, which changes with water content, and outputs a voltage that falls as soil gets wetter (capacitive types resist corrosion better than the older resistive probes).

Pins

AOUT (analog), VCC, GND.

Wiring notes

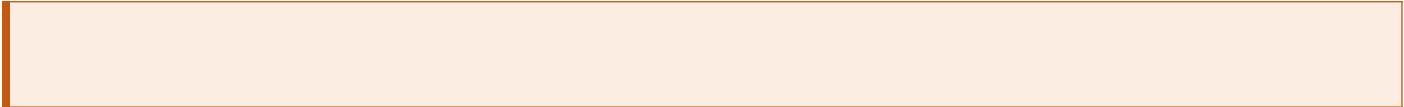
AOUT to an analog pin; VCC to 5V (or 3.3V per module); GND to GND.

Library required

None.

```
int moisture = analogRead(A0); // lower = wetter, on most capacitive modules
```

Testing: Read in dry air, then dip in water, and note both extremes before choosing a threshold.



Common errors

Assuming higher number always means wetter this is reversed on many capacitive modules; always calibrate.

Real-world applications: Automatic irrigation, greenhouse automation, plant-care reminders.



STUDENT CHALLENGE

Build a small 'sensor dashboard' sketch that reads the LDR, a push button, and the potentiometer in the same loop() and prints all three labelled values every second.

For any two sensors above, write down the exact value you measured in three different real conditions (e.g. bright/dim/dark for the LDR) before choosing a threshold.

Explain, in your own words, why an IR obstacle sensor and an ultrasonic sensor might disagree about whether an object is 'there' think about surface colour and material.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: One set of each sensor above (can be shared/rotated between groups); breadboards; Arduino Unos.

Preparation checklist: Pre-test every sensor kit before class a dead sensor wastes far more class time than it costs to check beforehand.

Demonstration procedure: Run the ultrasonic sensor live, printing raw pulseIn() duration next to the calculated distance, so students see the maths, not just the final number.

Questions to ask students: What physical quantity does each sensor actually measure, and what electrical signal does it turn that into? Why must every analog sensor be calibrated in its own environment?

Expected mistakes: Trusting a sensor's datasheet number over what the classroom actually measures; wiring VCC and GND reversed (some sensors survive this, some do not always double-check before power-up).

Assessment criteria: Student can correctly wire, code, and calibrate at least three different sensor types independently.

LEVEL 9 Actuators (Week 10)

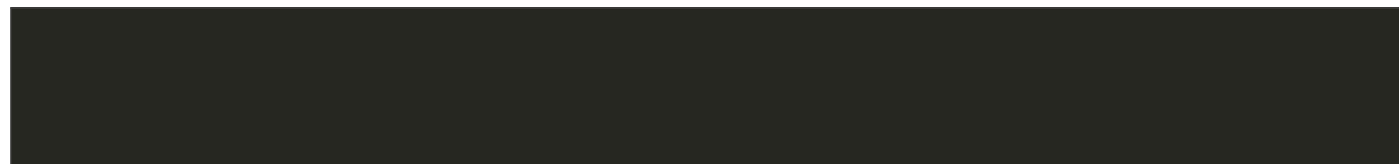
Real-life problem

Sensing a problem is useless if nothing happens in response. An actuator is any device that converts an electrical signal into physical action motion, sound, light, or flow. This week completes the SENSE → THINK → ACT model by adding the ACT half.

Push button, LDR, ultrasonic, IR, DHT, soil moisture

LED, buzzer, servo, DC motor, relay, pump

Buzzer



```
const int BUZZER_PIN = 8; tone(BUZZER_PIN, 1000); // 1000 Hz tone delay(500);  
noTone(BUZZER_PIN);
```

A passive buzzer needs tone()/noTone() to generate a note; an active buzzer only needs digitalWrite(HIGH/LOW).

Servo Motor

A servo motor moves to a specific angle (typically 0°-180°) and holds it ideal for a scanning ultrasonic mount, a feeder gate, or a kicking mechanism.



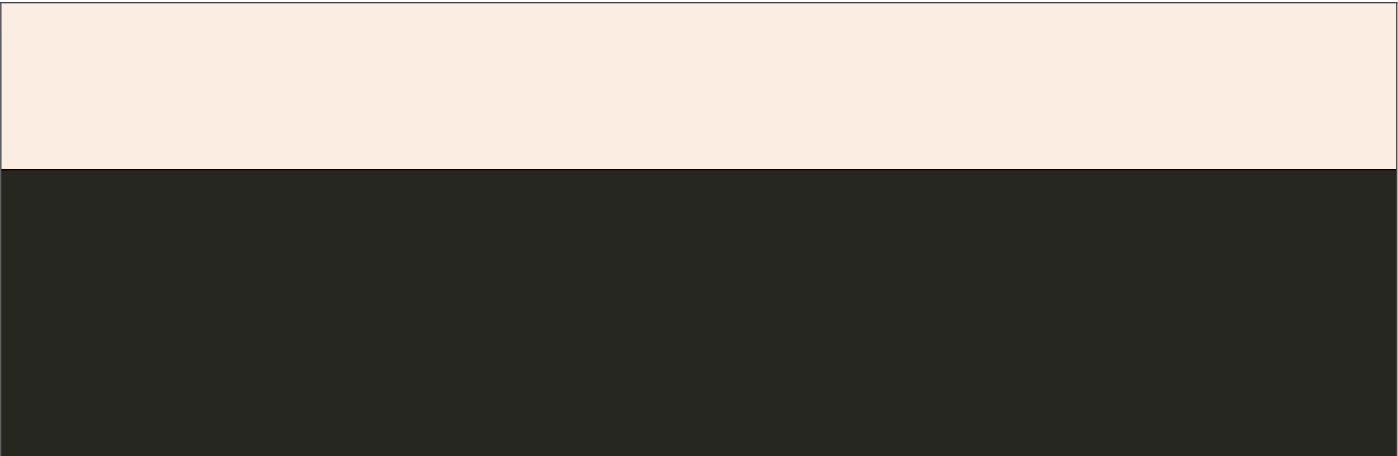
```
#include <Servo.h> Servo myServo;

void setup() { myServo.attach(9);
}

void loop() {
myServo.write(0); delay(1000); myServo.write(90); delay(1000); myServo.write(180);
delay(1000);
}
```

DC Motors and why they need a motor driver

An Arduino GPIO pin can safely supply only a few tens of milliamps. A DC motor can draw hundreds of milliamps to several amps when starting or under load far more than a pin can provide, and a motor's back-EMF can damage the microcontroller entirely. A motor driver (such as the L298N) sits between the Arduino and the motors: the Arduino sends small logic signals to the driver, and the driver switches the motor's own separate, higher-current power supply.

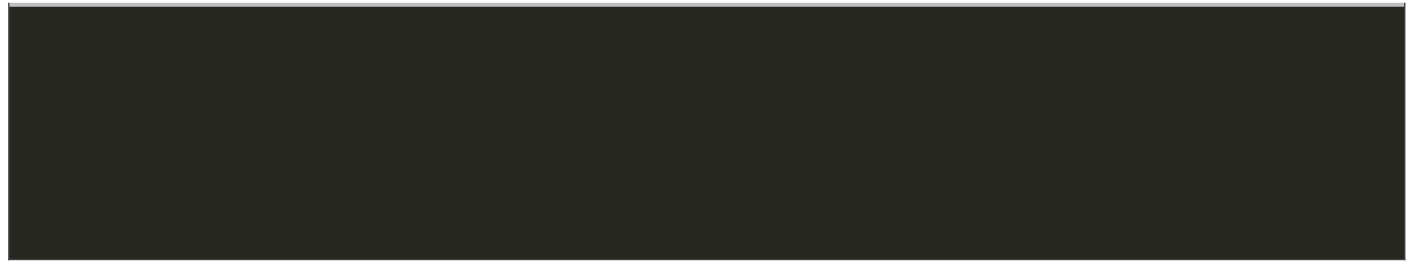


Why a motor driver is required

Never connect a DC motor directly to an Arduino pin. The motor draws too much current for the pin, and switching it off generates a voltage spike that can destroy the board. A motor driver (H-bridge) isolates and amplifies the control signal safely, and also allows the motor to be reversed by swapping which of its two terminals is driven HIGH.

```
// L298N motor driver - one motor channel const int IN1 = 4;
const int IN2 = 5;
const int ENA = 6; // PWM speed control
```

```
void motorForward(int speed) { digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
```



```
  analogWrite(ENA, speed); // 0-255
```

```
}
```

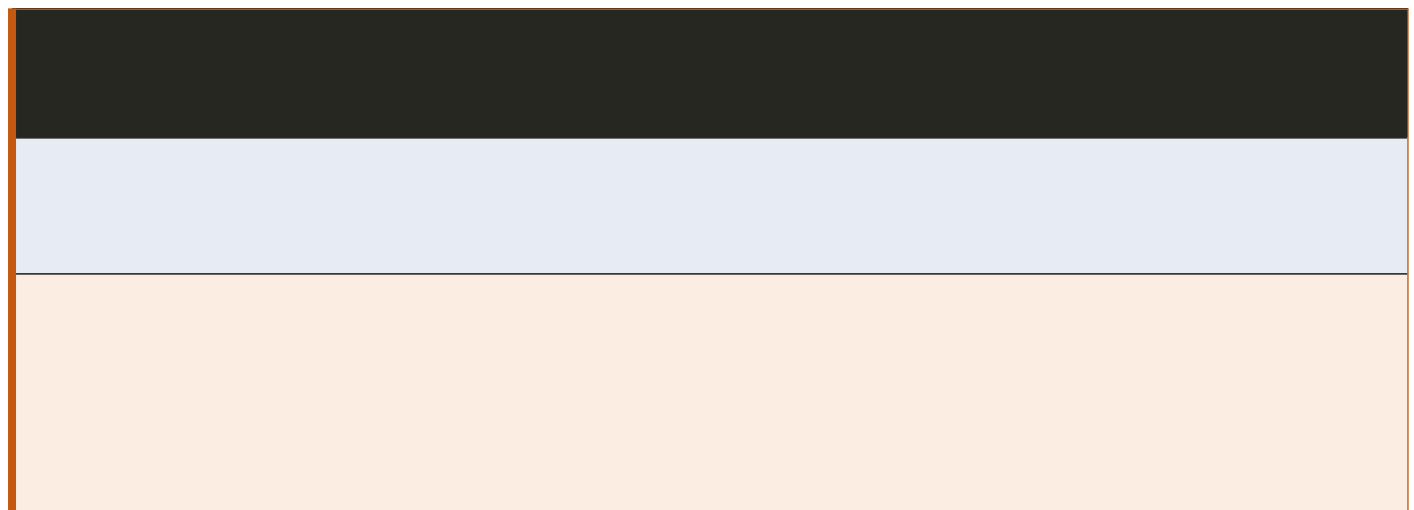
```
void motorStop() { digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
```

```
}
```

PWM (Pulse Width Modulation, via `analogWrite()`) rapidly switches a pin on and off; the ratio of on-time to off-time controls the average power delivered this is how speed and LED brightness are controlled without true analog voltage output.

Relay Module

A relay is an electrically controlled mechanical (or solid-state) switch. It lets a low-power microcontroller pin safely switch a completely separate, often much higher-power/higher-voltage circuit such as a water pump, a lamp, or a mains appliance without ever connecting that circuit's power directly to the Arduino.



```
const int RELAY_PIN = 7;
```

```
digitalWrite(RELAY_PIN, HIGH); // turns the relay (and whatever it switches) ON
```

```
// Most relay modules are active-LOW: check your specific module!
```

Safety

Any circuit powered from mains electricity (110V/220V) must only be wired by a qualified adult, never by students. For this course, relays should only switch low-voltage DC loads such as small pumps and lamps.

STUDENT CHALLENGE

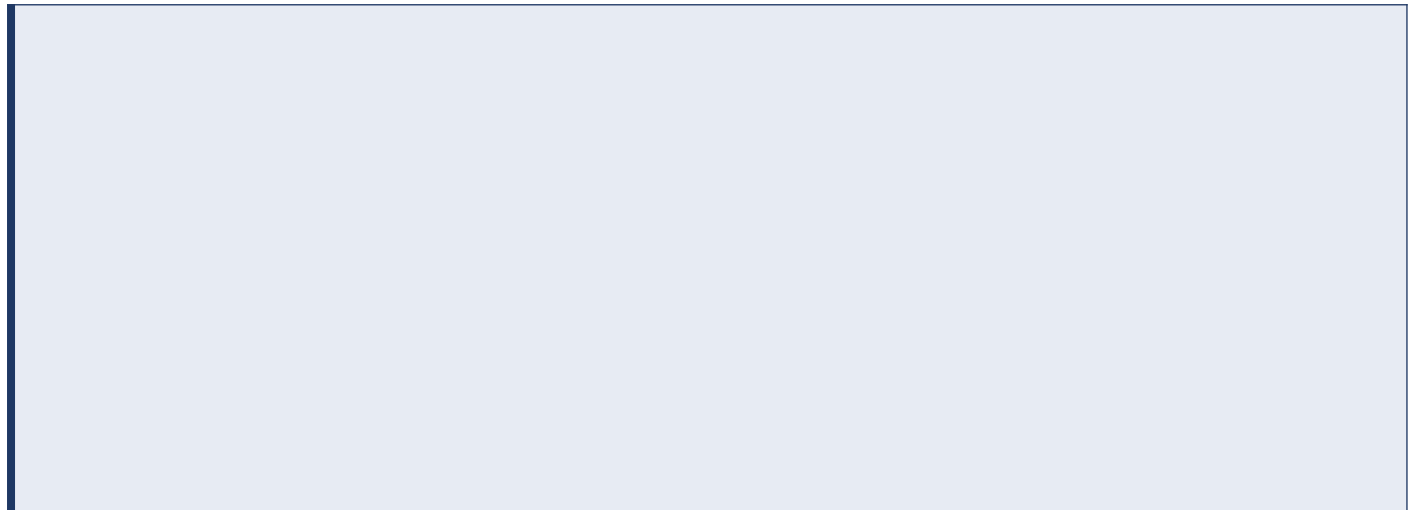
Build a 'push-button alarm': button press turns on a buzzer AND an LED at the same time using two separate `digitalWrite()` calls in the same block.

Sweep a servo automatically from 0° to 180° and back using a for loop instead of three separate `write()` calls.

Control DC motor speed with a potentiometer: read `analogRead(A0)`, `map()` it to 0-255, and feed it into `analogWrite()` on ENA.

Mini Project *Sensor-Triggered Actuator*

Combine any one sensor from Level 8 with any one actuator above using the full SENSE – THINK – ACT pattern: e.g. LDR senses darkness – if/else THINKS – servo or LED ACTS. Students choose their own sensor/actuator pair.



GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Buzzers, servo motors, an L298N motor driver + 2 DC motors for demonstration, a relay module + small 5V pump or lamp for demonstration.

Preparation checklist: Test the motor driver and a spare motor beforehand; have a spare fuse/driver board on hand, since incorrect wiring can damage a driver.

Demonstration procedure: Show a motor connected directly to a pin (using a spare, expendable pin/board or a simulator) briefly discussing the risk, then show the same motor correctly wired through the L298N.

Questions to ask students: Why can't a motor be driven directly from a digital pin? What does PWM actually do to a signal? Why is a relay needed instead of just a bigger transistor sometimes?

Expected mistakes: Wiring motor driver logic pins to the wrong Arduino pins; supplying motor power from the Arduino's 5V pin instead of a separate battery; forgetting a common GND between the Arduino and the motor driver's power supply.

Assessment criteria: Student demonstrates one working sensor-to-actuator chain and can explain, out loud, which part senses, which part thinks, and which part acts.

PROJECT 4 Automatic Farm Irrigation System (Week 11)

Problem the real-world problem we are solving

Crops die from under-watering, and growers waste time and water manually checking soil. An automatic irrigation system should water the soil only when it is actually dry, without a human checking it every day.

Learning objectives

- Read and calibrate an analog soil moisture sensor.
- Control a relay/pump from a threshold decision.

Apply timing so the pump does not repeatedly switch on and off ('chattering'). Understand safe power separation between the Arduino and the pump.

Components (Bill of Materials)

[illegible]

Arduino Uno or ESP32

1

Main controller

Capacitive soil moisture sensor

1

Detects soil dryness

Relay module (or logic-level MOSFET)

1

Switches the pump safely

Mini water pump (5V/12V submersible)

1

Delivers water

Water reservoir + tubing

1 set

Water storage and delivery

External power supply (matching pump voltage)

1

Powers the pump never from the Arduino

Breadboard + jumper wires

1 set

Assembly

OLED display (optional)

1

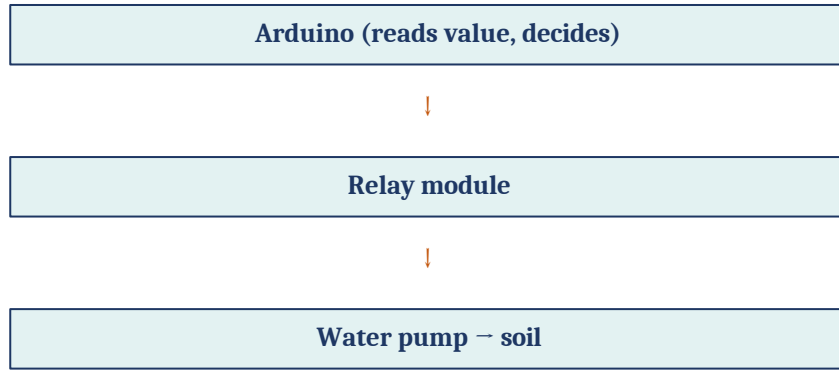
Shows live moisture % and pump status

Power note

The pump must be powered from its own external supply, switched by the relay. Never power a pump from the Arduino's 5V pin pumps draw far more current than the board can safely provide.

System block diagram





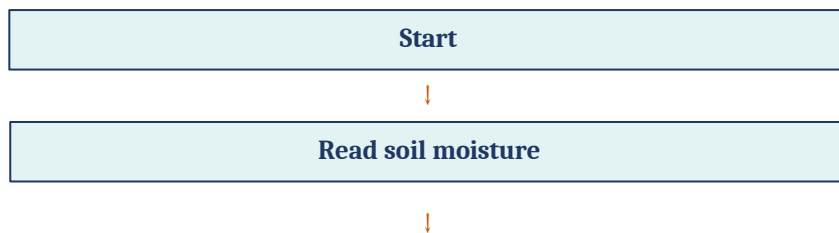
How the system works

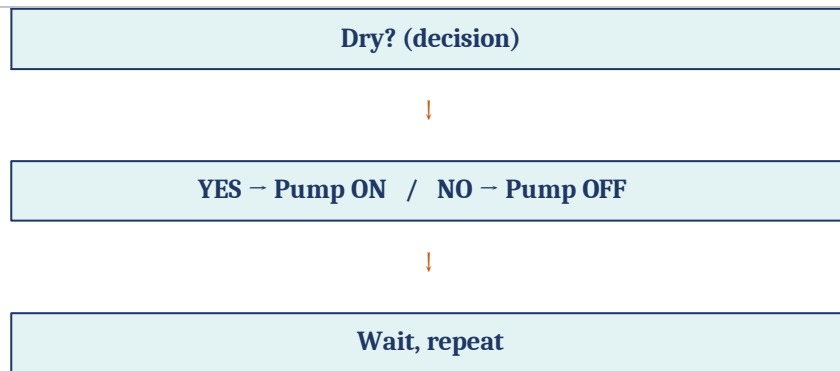
The Arduino continuously reads the soil moisture sensor. If the reading shows the soil is drier than a calibrated threshold, the Arduino switches the relay ON, which powers the pump from its own supply. Once the soil is moist enough, the relay switches OFF. A short delay between checks (and, optionally, a minimum pump run-time) prevents the pump rapidly clicking on and off at the threshold boundary.

Algorithm

Read the soil moisture sensor value.
Compare the value against the calibrated dry threshold.
If soil is dry: turn the pump ON.
If soil is moist enough: turn the pump OFF.
Wait briefly, then repeat from step 1.
(Optional) Log/display the moisture percentage and pump state.

Flowchart





Build steps

Wire the soil moisture sensor to an analog pin; confirm readings change between dry air and water.

Wire the relay's control pin to a digital pin, and the relay's switched side to the pump's external power circuit. Arduino and pump share only a common GND, never the same power rail.

Test the relay alone with `digitalWrite()`, listening/watching for the click, before connecting the pump.

Connect the pump and test it manually with the relay before adding sensor logic.

Add the moisture-reading and threshold logic.

Calibrate the threshold using real soil, not just air/water extremes.

Add timing so decisions are re-checked every few seconds, not continuously.

Version 1 Read and print moisture only

Confirm the sensor and wiring work before touching the pump.

```
const int MOISTURE_PIN = A0;

void setup() { Serial.begin(9600);
}

void loop() {
  int moisture = analogRead(MOISTURE_PIN); Serial.println(moisture);
  delay(500);
}
```

Version 2 Add the relay decision

Now the pump responds to the sensor.



```
const int MOISTURE_PIN = A0; const int RELAY_PIN = 7;

const int DRY_THRESHOLD = 600; // calibrate with real soil!

void setup() { pinMode(RELAY_PIN, OUTPUT);
Serial.begin(9600);
}

void loop() {
int moisture = analogRead(MOISTURE_PIN); Serial.println(moisture);

if (moisture > DRY_THRESHOLD) { digitalWrite(RELAY_PIN, HIGH); // dry -> pump ON
} else {
digitalWrite(RELAY_PIN, LOW); // moist -> pump OFF
}
delay(2000);
}
```

Final Version Add a minimum pump run-time and percentage display

Prevents rapid clicking at the threshold and gives readable feedback.



```

const int MOISTURE_PIN = A0; const int RELAY_PIN = 7;

const int DRY_THRESHOLD = 600;

const unsigned long PUMP_RUN_TIME = 5000; // ms, minimum time once started

void setup() { pinMode(RELAY_PIN, OUTPUT);
Serial.begin(9600);
}

void loop() {

int moisture = analogRead(MOISTURE_PIN);

int percent = map(moisture, 1023, 300, 0, 100); // calibrate these two raw values
Serial.print("Moisture: "); Serial.print(percent); Serial.println("");

if (moisture > DRY_THRESHOLD) { digitalWrite(RELAY_PIN, HIGH); delay(PUMP_RUN_TIME);
} else {
digitalWrite(RELAY_PIN, LOW);
}

delay(2000);
}

```

Code explanation key lines

map(moisture, 1023, 300, 0, 100) converts the raw calibrated range into a friendlier 0-100% scale the two raw numbers must come from your own sensor's real dry/wet readings.

PUMP_RUN_TIME forces the pump to run a sensible minimum duration once triggered, avoiding rapid on/off chattering right at the threshold.

The relay pin logic (HIGH/LOW = ON/OFF) depends on the specific relay module some are active-LOW; always verify with a simple test before wiring the pump.

Testing procedure

Test the sensor alone in air and water; record both raw values.

Test the relay alone (listen for the click) before connecting the pump.

Test the full system in dry soil, confirming the pump starts.

Water the soil manually and confirm the pump stops once moist.

Common problems

Pump never turns on

Threshold set wrong for this sensor/soil; relay wired active-LOW but coded active-HIGH

Pump never turns off

Threshold set too high; sensor reading drifting due to mineral buildup on probe

Relay clicks rapidly

No minimum run-time or hysteresis around the threshold

Debugging checklist

Is the sensor giving different readings in dry vs wet conditions? Confirm with raw Serial output first.

Does the relay click when tested alone with digitalWrite(), before the pump is attached?

Are the Arduino and pump power supplies sharing a common GND?

Is the threshold based on this classroom's actual soil, not a textbook number?

STUDENT CHALLENGE

Add an LED that lights whenever the pump is running, so the system's state is visible without the Serial Monitor. Add a manual override button that forces the pump ON for testing regardless of moisture.

Add a simple 'watering log': increment and print a counter each time the pump switches on.

Extension ideas

Add the OLED display to show moisture % and pump status permanently.

Add a rain sensor to skip watering if it's already raining.

Move the logic to an ESP32 and send moisture data to a simple web page.

Add a float switch in the reservoir to prevent the pump running dry.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: 1 kit per group: Arduino, soil sensor, relay, pump, external power supply, container of real soil. Preparation checklist: Pre-fill soil containers a day ahead so both 'dry' and 'freshly watered' states are available to test against in class.

Demonstration procedure: Deliberately power the pump from the Arduino's 5V pin briefly on a spare board (or explain via diagram) to show why this is unsafe, before demonstrating the correct external-supply wiring.

Questions to ask students: Why can't the pump share the Arduino's power pin? What would happen if the threshold were set far too high or far too low?

Expected mistakes: Sharing motor/pump power with the Arduino; not re-testing the threshold in the classroom's actual soil; missing common GND between the two power domains.

Assessment criteria: System correctly starts and stops the pump based on real soil moisture, and the student can explain their chosen threshold and how they calibrated it.

PROJECT 5 Automatic Animal / Poultry Feeder (Week 12)

Problem the real-world problem we are solving

Farm animals need feeding at consistent times, but a caretaker cannot always be present. An automatic feeder dispenses a controlled amount of feed at set times without a human physically opening the hopper.

Learning objectives

- Use a Real-Time Clock (RTC) module to keep accurate time independent of a computer.
- Trigger a servo-based dispenser at scheduled times.
- Organise scheduling and dispensing logic into clear functions.
- Give physical/audible feedback that feeding occurred.

Components (Bill of Materials)

[illegible]

Arduino Uno or ESP32

1

Main controller

DS3231 RTC module

1

Keeps accurate real time, even when powered off

Servo motor

1

Opens/closes the feed gate

Feed container / hopper with a gate

1

Holds and releases feed

Buzzer

1

Confirms feeding occurred

Push buttons (2-3)

Set

Manual feed / set time

LCD or OLED display (optional)

1

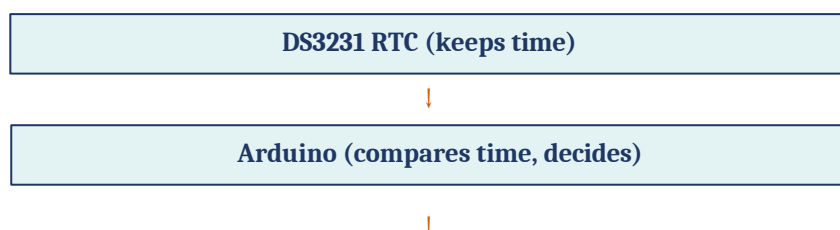
Shows current time and next feed time

Battery/power supply

1

Powers the system

System block diagram



Servo (opens gate) + Buzzer (confirms)

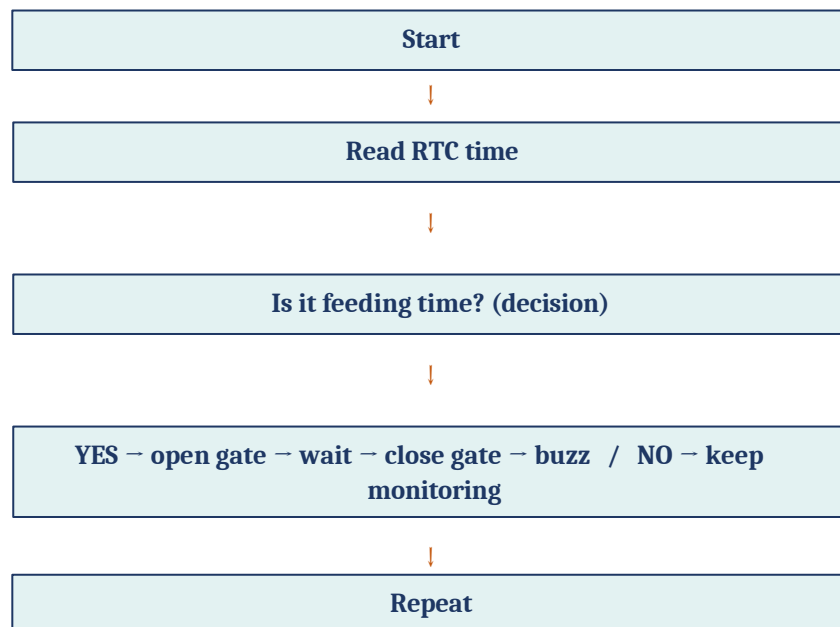
How the system works

The RTC module keeps accurate time using its own battery, even when the Arduino is unpowered. The Arduino continually reads the current time from the RTC and compares it against one or more stored feeding times. When the current time matches a feeding time, the Arduino rotates the servo to open the hopper gate, waits briefly for feed to fall through, closes the gate, and sounds the buzzer to confirm.

Algorithm

Read the current time from the RTC.
Compare the current time to the scheduled feeding time(s).
If it is feeding time: open the dispenser (servo).
Wait a set duration for feed to dispense.
Close the dispenser.
Sound the buzzer / indicate feeding occurred.
Continue monitoring the clock.

Flowchart



Build steps

Wire and test the RTC alone: print the time every second to confirm it keeps accurate time.
Set the RTC's time once (a one-time upload) after that it keeps time on its own battery.
Wire and test the servo alone, confirming open/close angles physically clear the hopper gate.
Add the time-comparison logic without touching the servo yet print 'FEEDING TIME!' when matched.
Connect the servo action to the matched condition.
Add the buzzer confirmation and a guard so feeding only triggers once per scheduled time, not repeatedly for the whole matching minute.

Version 1 Read and print RTC time



```
#include <Wire.h> #include <RTClib.h> RTC_DS3231 rtc;
```

```
void setup() { Serial.begin(9600); rtc.begin();  
}
```

```
void loop() {  
  DateTime now = rtc.now();  
  Serial.print(now.hour()); Serial.print(":"); Serial.println(now.minute()); delay(1000);  
}
```



```
}
```

Version 2 Add feeding-time comparison



```
#include <Wire.h> #include <RTClib.h> #include <Servo.h> RTC_DS3231 rtc;
```

```

Servo gateServo;

const int FEED_HOUR = 7; const int FEED_MINUTE = 0;

void setup() { Serial.begin(9600); rtc.begin(); gateServo.attach(9); gateServo.write(0); //
closed

}

void loop() {

DateTime now = rtc.now();

if (now.hour() == FEED_HOUR && now.minute() == FEED_MINUTE) { Serial.println("FEEDING
TIME!");

gateServo.write(90); // open delay(3000); gateServo.write(0); // close

}

delay(1000);

}

```

Learning Objectives

- Understand the difference between hardware, software, and algorithms in embedded systems
- Draw and interpret simple flowcharts that represent real-world processes
- Recognise the Input-Processing-Output pattern in everyday devices
- Write and execute your first C++ program with confidence

Final Version Prevent repeated feeding within the same minute + buzzer + function organisation

Real engineering code separates concerns into functions.

```
#include <Wire.h> #include <RTClib.h> #include <Servo.h> RTC_DS3231 rtc;

Servo gateServo;

const int BUZZER_PIN = 8; const int FEED_HOUR = 7; const int FEED_MINUTE = 0; bool fedToday =
false;

void dispenseFeed() { gateServo.write(90); delay(3000); gateServo.write(0); tone(BUZZER_PIN,
1000, 500);
}

void setup() { Serial.begin(9600); rtc.begin(); gateServo.attach(9); gateServo.write(0);
}

void loop() {
DateTime now = rtc.now();

bool isFeedTime = (now.hour() == FEED_HOUR && now.minute() == FEED_MINUTE);
```



```
if (isFeedTime && !fedToday) { dispenseFeed();  
fedToday = true;  
}  
if (!isFeedTime) {  
fedToday = false; // reset once the minute has passed  
}  
delay(1000);  
}
```

Code explanation key lines

fedToday is a guard flag: without it, the servo would trigger on every loop() pass for the entire 60 seconds that the minute matches.

dispenseFeed() is a function because 'dispense feed' is one clear action reused for every scheduled time, including ones a student may add later.

The RTC keeps time using its own coin-cell battery, so time is not lost even if the Arduino is unplugged or reset.

Testing procedure

Set the RTC's time and confirm it prints correctly and keeps ticking.
Temporarily set FEED_HOUR/FEED_MINUTE to a time one minute in the future and confirm the servo fires exactly once.
Confirm fedToday correctly resets so the next day's feeding also works.

Common problems

- Feeds repeatedly for a full minute
- Missing fedToday guard flag
- RTC time is wrong/resets
- Coin-cell battery missing or dead; time-setting code left in loop() instead of a one-time setup

Servo doesn't reach the feed

Gate mechanism geometry not matched to servo angle range adjust angles physically, not just in code

Debugging checklist

- Does the RTC alone print correct, ticking time before any servo logic is added?
- Does the servo alone open and close the physical gate fully when tested with fixed angles?
- Is there a guard preventing repeated triggering within the same matching minute?
- Is the RTC battery installed and fresh?

STUDENT CHALLENGE

Add a second feeding time later in the day using an array of times instead of one fixed hour/minute. Add a manual 'feed now' push button for emergencies.

Display the next feeding time on an LCD/OLED, updating every second.

Extension ideas

- Log every feeding event with a timestamp to an SD card.
- Add a load cell to confirm feed actually left the hopper (detect an empty hopper).
- Move to ESP32 and send a notification when feeding occurs.
- Add multiple hoppers/gates for different feed types.

GUIDE

Suggested time: 4 hours (2 classes in a week) + Assessment

Equipment needed: 1 RTC + Arduino + servo + buzzer per group; a small feed-safe container to model the hopper (does not need real feed for testing).

Preparation checklist: Set every RTC's time once as a class, well before students start their own logic, since incorrect time silently breaks the whole project.

Demonstration procedure: Set a feeding time one minute ahead in front of the whole class and count down to the exact moment the servo fires makes the RTC concept concrete.

Questions to ask students: Why does the system need `fedToday` if the condition itself looks correct? What would happen to feeding if the RTC lost power and had no coin-cell battery?

Expected mistakes: Forgetting the guard flag and dispensing feed repeatedly for a full minute; setting the RTC time inside loop() so it keeps resetting.

Assessment criteria: Independent modification: student adds a second feeding time and demonstrates both triggering correctly on their own schedule.

PHASE 4 Robotics Fundamentals (Weeks 13-16, 16 hours)

This is where it gets exciting. Your sensors and logic now power a moving robot. Everything you have learned comes together here.

This is the phase most students have been waiting for: building a robot that moves. Every idea from Phases 1-3 variables, decisions, loops, functions, sensors, actuators, PWM comes together here. By the end of Phase 4, every student or team will have a working, autonomous obstacle-avoiding robot.

LEVEL 10 Robotics Fundamentals (Week 13)

Real-life problem

Turning a stationary Arduino project into a robot that moves around a room requires a chassis, wheels, a way to drive those wheels, and a way to carry its own power none of which existed in earlier projects, which all sat still on a bench.

The parts of a mobile robot

Chassis

The physical frame/body the robot is built on.

Wheels + DC geared motors

Convert electrical power into motion. 'Geared' motors trade top speed for usable torque.

Caster wheel

A free-spinning wheel (often at the front or back) that lets a 2-wheel-drive robot balance and turn without a third motor.

Motor driver (e.g. L298N)

Safely lets the microcontroller control high-current motors (see Level 9).

Battery pack

Powers both the motors (high current) and, often separately regulated, the microcontroller.

Microcontroller (Arduino/ESP32)

Runs the decision-making code.

Sensors (ultrasonic, IR, etc.)

Give the robot awareness of its surroundings.

Power considerations

Motors and logic circuits often need different voltages a separate battery pack for motors, regulated down (or via a separate 5V source) for the Arduino, is common and safer.

A robot's battery must supply enough current for both motors under load, not just their 'no-load' rating undersized batteries cause the Arduino to brown-out and reset mid-motion.

Always share a common GND between every power source and the microcontroller, even when their positive supplies are separate.

Why not connect motors directly to Arduino pins?

Covered fully in Level 9: motors draw far more current than a GPIO pin can supply, and switching them generates voltage spikes that can damage the board. A 2WD robot always uses a motor driver between the Arduino and its motors.

Assembly overview: building the 2WD platform

Attach the two DC geared motors to the chassis using the supplied brackets/screws.

Fit wheels to the motor shafts.

Fit the caster wheel to balance the chassis (usually at the opposite end from the motors, or centred underneath).

Mount the motor driver board and Arduino on the chassis deck, keeping motor wiring short.

Mount the battery pack securely check its weight does not unbalance the chassis.

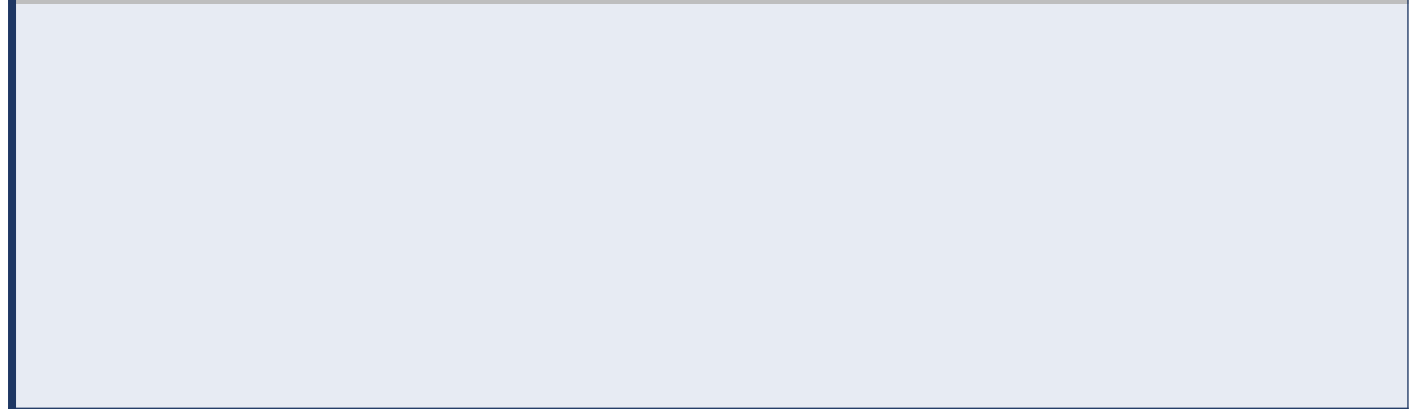
Wire both motors to the motor driver's two output channels, and the driver's logic pins to the Arduino.

STUDENT CHALLENGE

Sketch your own robot's power diagram, showing every component's power source and where all the GND wires meet. Research and write down the stall current of your specific DC motors what does this tell you about the battery you need?

Before wiring the driver, predict on paper which two Arduino pins you will use for each motor's direction, and which pins for PWM speed.

GUIDE



Suggested time: 4 hours (2 classes)

Equipment needed: One 2WD chassis kit per group (motors, wheels, caster, screws); motor driver; battery pack. Preparation checklist: Pre-check every chassis kit is complete missing screws or a dead motor stalls an entire group.

Demonstration procedure: Assemble one chassis live, narrating each mechanical decision (motor placement, caster position, weight balance) before students build their own.

Questions to ask students: Why does the caster wheel not need a motor? What could go wrong if the battery is mounted unevenly on the chassis?

Expected mistakes: Motors mounted so wheels rub the chassis; caster wheel positioned so the robot tips; wiring the driver's logic and power sections to the same supply.

Assessment criteria: Chassis is fully and correctly assembled, motors spin freely by hand, and the student can explain the power path from battery to wheel.

Programming Robot Movement (Week 14)

Real-life problem

A robot needs to reliably move forward, reverse, turn, and stop and its code needs to say so in a way that reads like a description of behaviour, not a maze of pin numbers.

Organising movement into functions

```
// Motor A (left) on IN1/IN2/ENA, Motor B (right) on IN3/IN4/ENB const int IN1 = 4, IN2 = 5,
ENA = 6;

const int IN3 = 7, IN4 = 8, ENB = 9;

void moveForward(int speed) {

digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW); digitalWrite(IN3, HIGH); digitalWrite(IN4,
LOW); analogWrite(ENA, speed); analogWrite(ENB, speed);

}

void moveBackward(int speed) {

digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH); digitalWrite(IN3, LOW); digitalWrite(IN4,
HIGH); analogWrite(ENA, speed); analogWrite(ENB, speed);

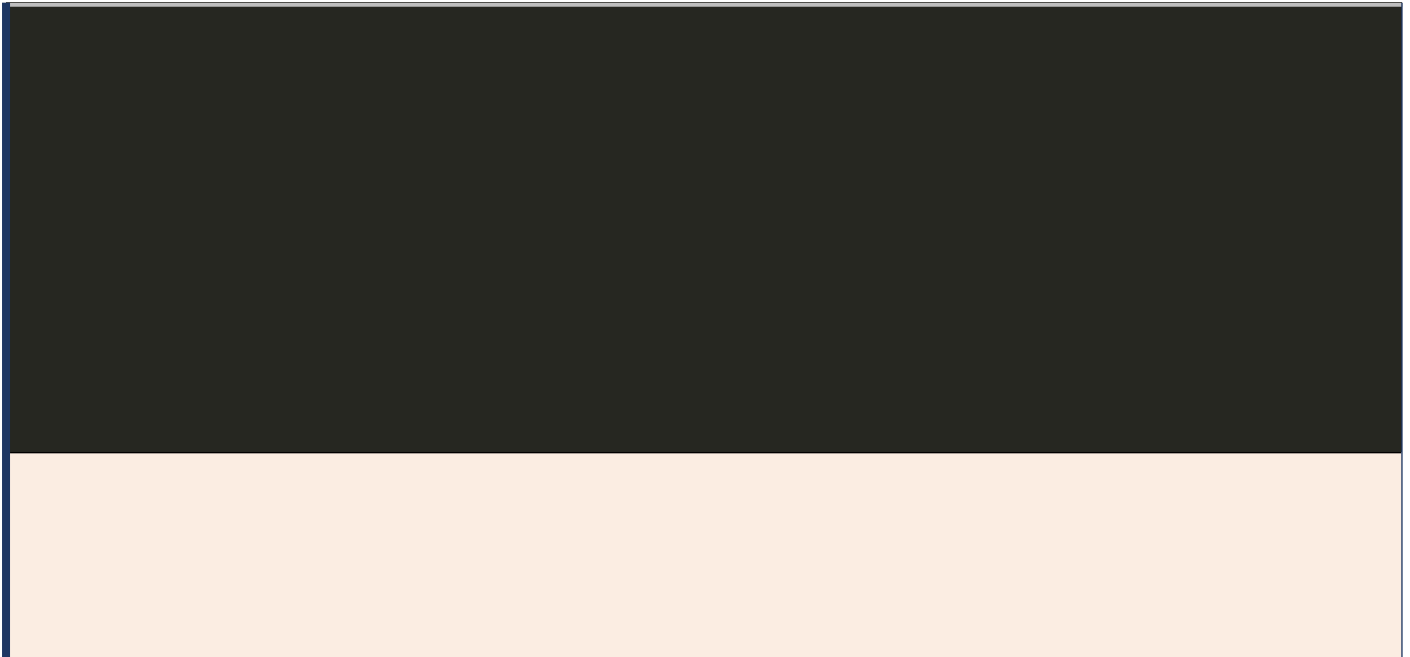
}
```

```
void turnLeft(int speed) {  
  
digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH); // left wheel back digitalWrite(IN3, HIGH);  
digitalWrite(IN4, LOW); // right wheel forward analogWrite(ENA, speed); analogWrite(ENB,  
speed);  
  
}  
  
void turnRight(int speed) {  
  
digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW); digitalWrite(IN3, LOW); digitalWrite(IN4,  
HIGH); analogWrite(ENA, speed); analogWrite(ENB, speed);  
  
}  
  
void stopRobot() {  
  
digitalWrite(IN1, LOW); digitalWrite(IN2, LOW); digitalWrite(IN3, LOW); digitalWrite(IN4,  
LOW);  
  
}
```

Why did we write it this way?

Every movement function takes speed as a parameter, so the same function drives slow, careful movement or fast movement without duplicating code. Once these five functions exist and are tested, every later project (obstacle avoidance, mopping, soccer) simply calls `moveForward(180)`; or `turnLeft(150)`; the low-level wiring detail is hidden behind a clear name.

Test sequence



```
void setup() {  
  
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT); pinMode(IN3, OUTPUT);  
  pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);  
  
}  
  
void loop() {  
  
  moveForward(180); delay(1000); stopRobot(); delay(500); turnRight(150);      delay(500);  
  stopRobot(); delay(500); moveBackward(180); delay(1000); stopRobot(); delay(2000);  
  
}
```

STUDENT CHALLENGE

If one wheel spins backward when you intended forward, fix it by swapping that motor's IN pins in code (not by re-wiring) explain why this works.

Add a moveForwardSlow() and moveForwardFast() using two constant speed values, calling moveForward() with each. Combine two turns and two forward moves to drive your robot in a square path, timing each segment.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Assembled 2WD chassis per group, propped on a stand (wheels off the ground) for safe initial testing.

Preparation checklist: Have every group test motors off the ground first a robot randomly driving off a bench during debugging is disruptive and can damage equipment.

Demonstration procedure: Deliberately wire one motor backwards, run moveForward(), and show the robot spinning instead of driving straight then fix it in code, not wiring, live.

Questions to ask students: Why do turnLeft() and turnRight() spin the two wheels in opposite directions instead of just stopping one wheel? What does the speed parameter actually change electrically?

Expected mistakes: Both motors wired identically but spinning opposite ways due to physical mounting; ENA/ENB pins not chosen as PWM-capable pins.

Assessment criteria: Robot correctly performs a called sequence of forward/backward/left/right/stop using named functions with a speed parameter.

PROJECT 2 2WD Remote-Controlled Car (Week 15)

Problem the real-world problem we are solving

Before a robot can make its own decisions, students must control it directly proving the mechanical and motor-control foundation works and is fully understood before autonomy is added.

Learning objectives

- Receive commands over Bluetooth and act on them.
- Reuse the movement functions built in Week 14 without modification.
- Understand serial communication between two devices.
- Practise structured testing of a physical system.

Components (Bill of Materials)

2WD chassis with motors (from Level 10)

1

Robot platform

Arduino Uno or compatible

1

Main controller

L298N motor driver

1

Motor control

HC-05 Bluetooth module

1

Wireless remote control link

Battery pack

1

Power

Jumper wires, breadboard (if needed)

Set

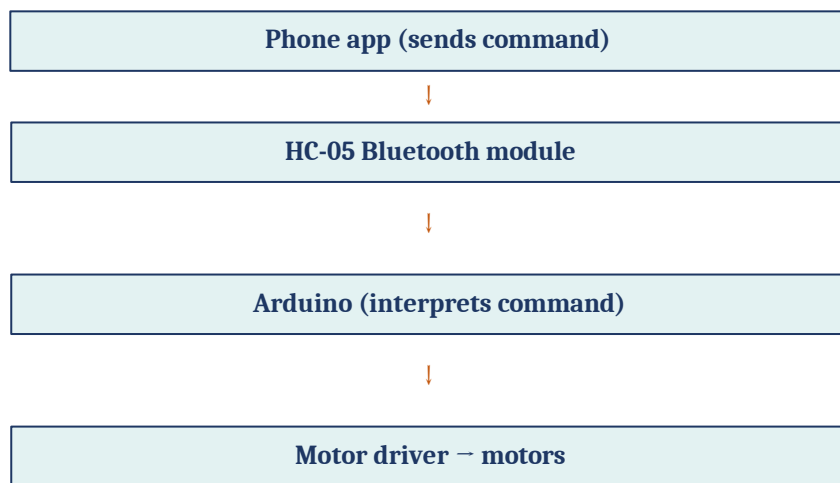
Assembly

Smartphone with a Bluetooth serial controller app

1

Sends drive commands

Circuit / system block diagram



How the system works

The HC-05 module receives characters sent wirelessly from a phone app over Bluetooth and makes them available to the Arduino exactly like data typed into the Serial Monitor. The Arduino reads each incoming character and calls the matching movement function (already built and tested in Week 14) the robot's mechanical behaviour does not change at all; only the source of the command changes, from code to a human's button press.

Algorithm

Wait for a character to arrive from Bluetooth.

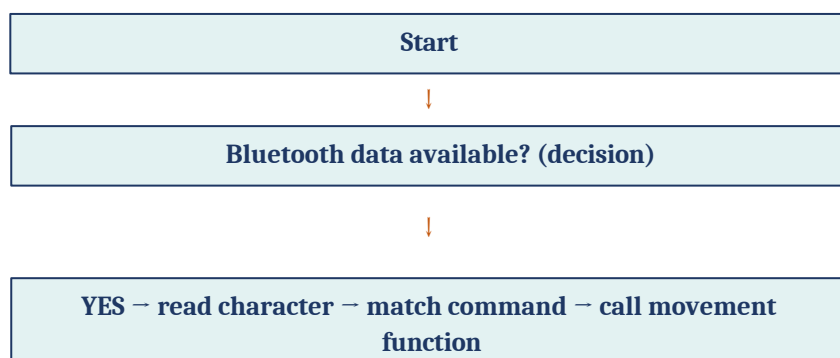
Read the character.

Match it against known commands (F = forward, B = backward, L = left, R = right, S = stop).

Call the matching movement function.

Repeat continuously.

Flowchart



↓

NO – keep waiting

↓

Repeat

Build steps

Wire the HC-05: VCC to 5V, GND to GND, TX to Arduino RX, RX to Arduino TX (through a voltage divider if the module is 3.3V logic).

Pair the HC-05 with a phone using a Bluetooth serial terminal/controller app.

Test communication alone: send a character from the phone and Serial.print() it back, with motors disconnected.

Connect the tested movement functions from Week 14.

Map each command character to a movement function call.

Test on the ground, at low speed first, in an open area.

Version 1 Confirm Bluetooth communication only

No motors yet prove the link works first.

```
void setup() {  
  Serial.begin(9600); // HC-05 default baud rate  
}  
  
void loop() {  
  if (Serial.available()) {  
    char command = Serial.read(); Serial.print("Received: "); Serial.println(command);  
  }  
}
```

Version 2 Map commands to movement functions

Reuses moveForward(), turnLeft(), etc. exactly as built in Week 14 no changes needed to those functions.

```

void loop() {
  if (Serial.available()) {
    char command = Serial.read(); switch (command) {
      case 'F': moveForward(180); break; case 'B': moveBackward(180); break; case 'L':
        turnLeft(150); break; case 'R': turnRight(150); break; case 'S': stopRobot(); break;
    }
  }
}

```

Final Version Add a safety timeout (auto-stop if signal is lost)

Prevents the robot driving away uncontrolled if Bluetooth disconnects mid-command.



```

unsigned long lastCommandTime = 0;
const unsigned long TIMEOUT = 1000; // ms

void loop() {
  if (Serial.available()) {
    char command = Serial.read(); lastCommandTime = millis(); switch (command) {
      case 'F': moveForward(180); break; case 'B': moveBackward(180); break; case 'L':
        turnLeft(150); break; case 'R': turnRight(150); break; case 'S': stopRobot(); break;
    }
  }
  if (millis() - lastCommandTime > TIMEOUT) { stopRobot(); // safety: no recent command received }
}

```

Code explanation key lines

`Serial.available()` checks whether new Bluetooth data has arrived before trying to read it, avoiding reading 'nothing.'

`millis()` returns milliseconds since the Arduino started comparing it to `lastCommandTime` detects a lost connection without ever using `delay()`, which would freeze command response.

The switch statement (from Level 3) maps each received character directly onto the correct, already-tested movement function.

Testing procedure

Test HC-05 pairing and raw character echo with motors disconnected.

Test each movement command individually at low speed with wheels off the ground.

Test on the floor in an open, obstacle-free space.

Test the safety timeout by closing the app mid-drive and confirming the robot stops.

Common problems

No Bluetooth connection

Wrong baud rate; TX/RX swapped; module not in pairing mode

Robot responds to wrong commands

Case-sensitivity mismatch between app buttons and switch cases

Robot keeps driving after app closes

Missing safety timeout logic

Debugging checklist

Does raw character echo work before any motor logic is added?

Are TX/RX genuinely crossed (module TX → Arduino RX, and vice versa)?

Does every case in the switch match exactly what the phone app actually sends (including case sensitivity)?

Is there a safety timeout, and does it actually trigger when tested?

STUDENT CHALLENGE

Add a variable speed command ('1' to '9') that changes the speed parameter used by subsequent movement commands.

Add a horn/buzzer command.

Log every received command with a timestamp to the Serial Monitor for a short debugging session.

Extension ideas

Replace the phone app's fixed buttons with a joystick-style control app that sends continuous direction+speed.

Add an LED that shows Bluetooth connection status.

Combine this project with an ultrasonic sensor to add an automatic 'panic stop' if the robot is about to hit something while under manual control.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: HC-05 module + paired smartphone per group; assembled 2WD robots from Weeks 13-14. Preparation checklist: Pre-pair every HC-05/phone combination the day before class Bluetooth pairing issues can consume an entire lesson if left to class time.

Demonstration procedure: Show the raw-character version first (Version 1), typing commands from a laptop terminal, before adding any motor logic isolates communication problems from motor problems.

Questions to ask students: What happens if `Serial.read()` is called with no data available? Why is a safety timeout necessary for a wireless-controlled robot specifically?

Expected mistakes: Confusing the module's TX/RX with the Arduino's TX/RX (they must cross, not match); leaving motors connected while debugging communication.

Assessment criteria: Robot responds correctly and promptly to all five basic commands from the phone app, and automatically stops if the connection is lost.

PROJECT 3 Obstacle-Avoiding Robot (Week 16 Phase Milestone)

Problem the real-world problem we are solving

A genuinely useful robot cannot rely on a human driver for every decision. This project gives the robot its first real autonomy: it senses its environment and decides its own path.

Learning objectives

Combine an ultrasonic sensor with a servo to scan surroundings.

Implement autonomous decision-making using if/else built on real sensor data.

Build the program in stages rather than writing it all at once.

Debug autonomous (unsupervised) behaviour systematically.

Components (Bill of Materials)

2WD chassis with motors and driver (from

1

Robot platform

Projects 2)

HC-SR04 ultrasonic sensor

1

Measures distance to obstacles

Servo motor

1

Sweeps the ultrasonic sensor left/right

Arduino Uno or compatible

1

Main controller

Battery pack

1

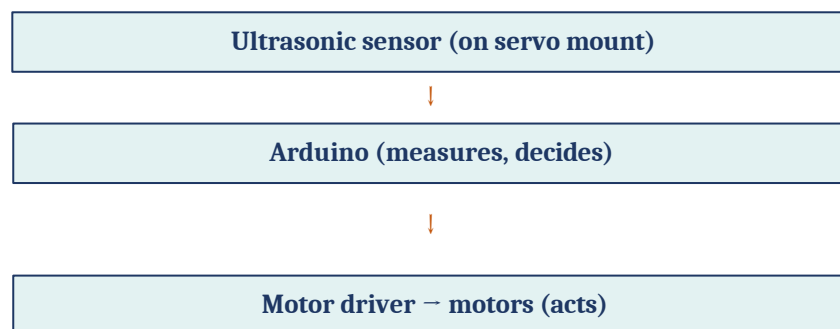
Power

Jumper wires

Set

Assembly

Circuit / system block diagram



How the system works

The robot drives forward while continuously measuring the distance ahead with the ultrasonic sensor. When an obstacle is detected within a safe distance, the robot stops, sweeps the servo-mounted sensor left and right to measure which direction is more open, turns toward the safer direction, and resumes moving forward. This is the SENSE → THINK → ACT loop running continuously and autonomously.

Algorithm

Move forward.

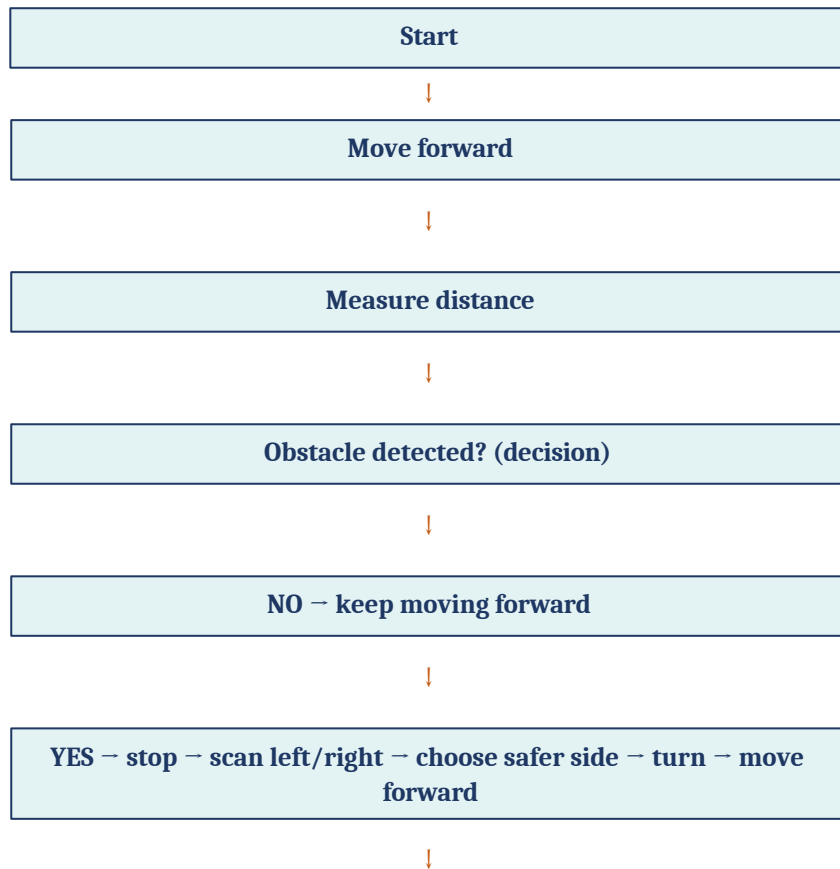
Measure distance ahead.

Is an obstacle within the safe distance? If not, repeat from step 1.

If yes: stop.

Scan left and measure distance; scan right and measure distance.
Choose the direction with more open space.
Turn briefly toward that direction.
Resume moving forward; repeat from step 1.

Flowchart



Repeat

Build steps

Mount the ultrasonic sensor on the servo so it can sweep left/right; mount both at the front of the chassis.

Test the ultrasonic sensor alone (Level 8) and the servo alone (Level 9) before combining them.

Combine sensor and servo: sweep and print distance at each angle.

Add the movement functions from Week 14 (already tested).

Build the obstacle logic in stages exactly as shown below do not skip to the final version.

Test with wheels on the ground in an open area with soft, safe obstacles.

Version 1 Move forward only, ignoring obstacles

Confirms driving works before any sensing logic is added.

```
void setup() {  
  setupMotors(); // pinMode calls from Week 14  
}  
  
void loop() {  
  moveForward(150);  
}
```

Version 2 Add sensing, but only stopping (no turning yet)

Students write this stage themselves in class do not hand it over pre-written.

```
const int SAFE_DISTANCE = 20; // cm  
  
void loop() {  
  long distance = readDistanceCM(); // from Level 8 if (distance > SAFE_DISTANCE) {  
  moveForward(150);
```

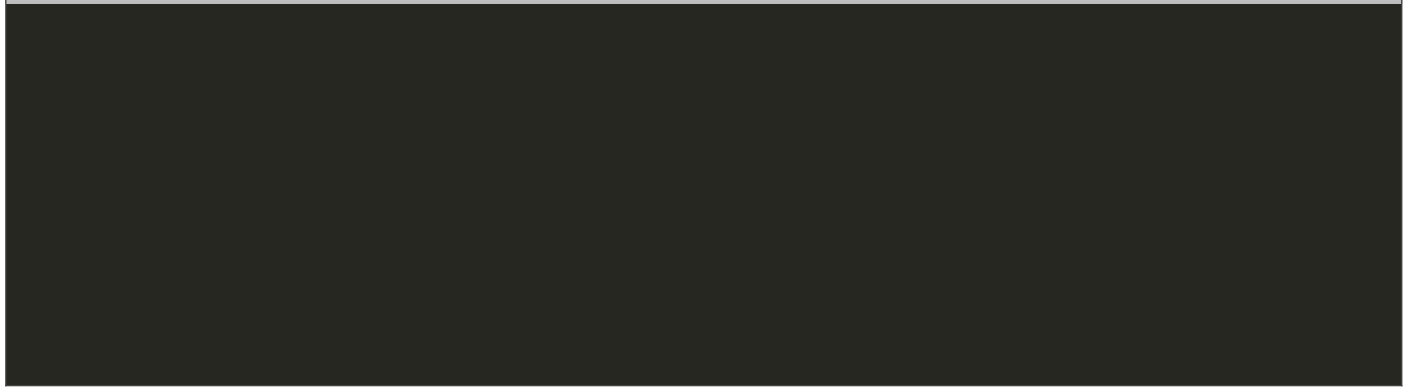
```
} else {  
  stopRobot();  
}  
}
```

Final Version Add scanning and turning toward the open side

Full autonomous behaviour.



```
const int SAFE_DISTANCE = 20;  
  
void loop() {  
  long distance = readDistanceCM();  
  
  if (distance > SAFE_DISTANCE) { moveForward(150);  
  } else {  
    stopRobot(); delay(200);  
  
    myServo.write(150); // look left delay(400);  
    long leftDistance = readDistanceCM();  
  
    myServo.write(30); // look right delay(400);  
    long rightDistance = readDistanceCM();  
  
    myServo.write(90); // centre again
```



```
delay(300);

if (leftDistance > rightDistance) { turnLeft(150);
} else {
  turnRight(150);
}
delay(400); stopRobot();
}
```

Code explanation key lines

The final version is only reached after Versions 1 and 2 work independently this is the 'build in stages' rule from Section 6 of the course philosophy applied to a real project.

myServo.write(150) / write(30) point the sensor left and right; the delay() after each lets the servo physically finish moving before a reading is taken.

Comparing leftDistance and rightDistance is the 'THINK' step: the robot decides its own direction based on real sensor data, not a pre-programmed turn.

Testing procedure

Test Version 1 with wheels raised off the ground.

Test Version 2 with a hand as a moving 'obstacle' at a known distance.

Test the full Version 3 in an open space with soft obstacles (cardboard boxes, not walls) in case of collision during debugging.

Test both a left-open and a right-open scenario to confirm the robot picks correctly each time.

Common problems

Robot stops but never turns

Missing or broken scan-and-compare block; SAFE_DISTANCE never true again

Robot turns into the obstacle

Left/right distance readings swapped, or servo angles for 'left' and 'right' reversed

Robot oscillates without progress

SAFE_DISTANCE too large relative to the room, causing constant stop-scan-turn cycles

Debugging checklist

Does readDistanceCM() alone give sensible, stable numbers when tested first?

Do left/right servo angles actually point the sensor where the code assumes they do check physically, not just in code?

Is SAFE_DISTANCE realistic for the room the robot is tested in?

Does the robot fully stop before scanning, or is it still moving (and therefore measuring a moving target)?

STUDENT CHALLENGE

Add a rule: if both left and right are also blocked, reverse briefly before scanning again.

Average three ultrasonic readings per angle instead of one, to reduce noise (reuse the array-averaging technique from Level 6).

Log every decision ('turned left, left=45cm right=20cm') to the Serial Monitor to help debug the robot's 'reasoning.'

Extension ideas

Add a rear ultrasonic sensor to detect obstacles while reversing.

Combine with the HC-05 from Project 2 so a human can override autonomy at any time.

Track how many turns the robot makes and print a simple 'confidence' score.

Enter it in the Week 20 Robotics Challenge for fastest/most reliable obstacle avoidance.

GUIDE

Suggested time: 4 hours (2 classes in a week) Milestone project

Equipment needed: Fully assembled 2WD robots with ultrasonic + servo mounted; open floor space cleared of fragile

objects.

Preparation checklist: Clear a safe test area; have soft/cardboard obstacles ready rather than walls or furniture. Demonstration procedure: Run Version 2 (stop-only) live and ask the class to predict, before you show it, what the robot will do when it meets an obstacle it can't yet turn away from.

Questions to ask students: Why do we build this in three versions instead of writing the final version directly? What would happen if the robot scanned while still moving forward?

Expected mistakes: Skipping straight to the final version and being unable to debug it when it fails; `SAFE_DISTANCE` copied from another team's robot without retesting.

Assessment criteria: MILESTONE: every student/team has a robot that autonomously detects an obstacle, chooses a sensible direction, and continues moving without human input.

PHASE 5 Advanced Robotics Projects (Weeks 17-20, 16 hours)

The foundation is solid. Now you specialise. You will build three completely different robots, each solving a different problem, using the same principles.

With a working autonomous robot as their foundation, students now specialise that same platform into three very different machines proving that the SENSE – THINK – ACT pattern and the movement functions from Phase 4 generalise far beyond obstacle avoidance.

Improving Autonomous Navigation (Week 17)

Real-life problem

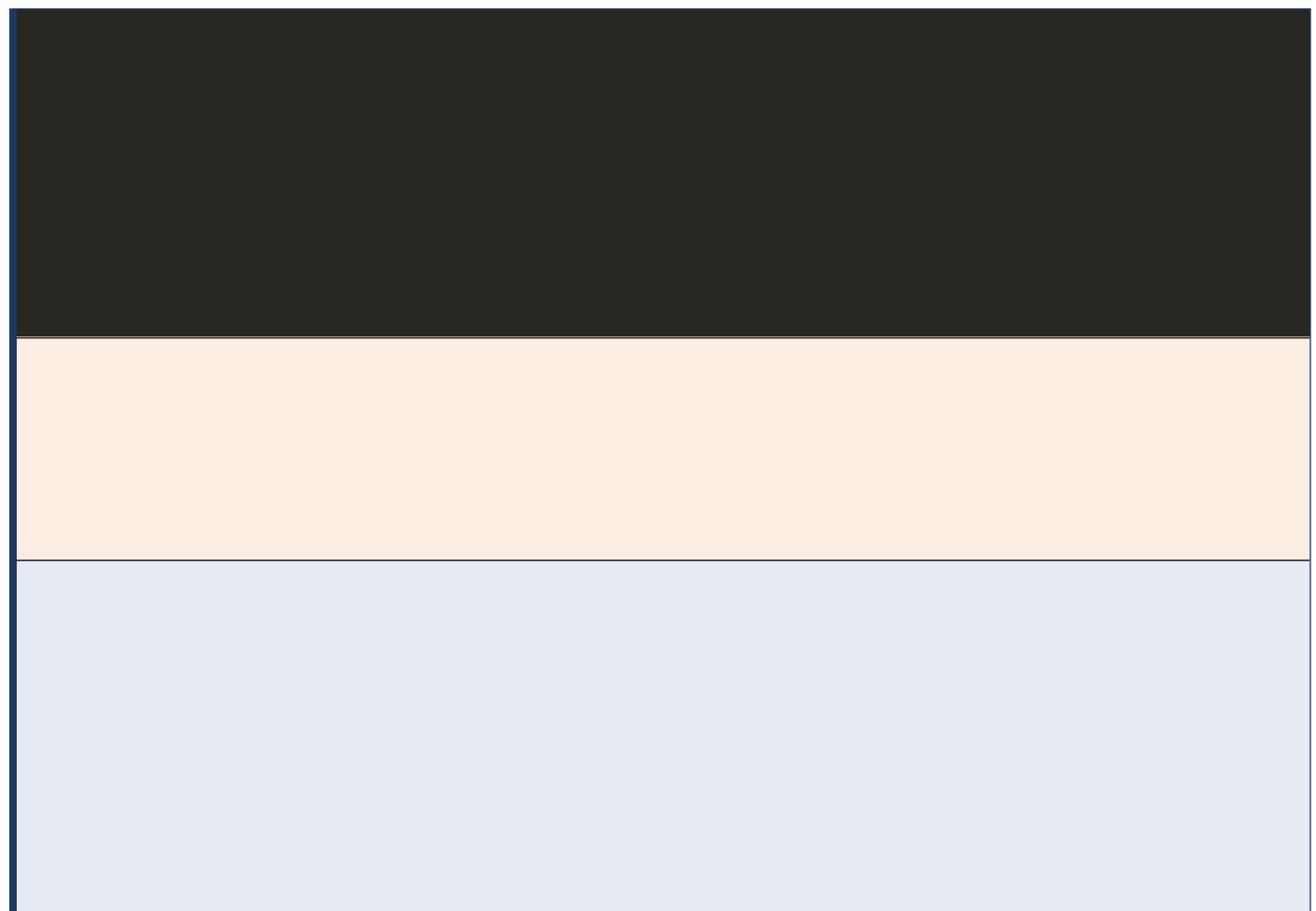
The Week 16 obstacle-avoider works, but real rooms expose its weaknesses: it may turn the same direction every time, get stuck oscillating in a corner, or turn too sharply and clip an obstacle it just avoided.

Improving the decision logic

Add a 'both sides blocked' case: if left and right distances are both small, reverse briefly before scanning again, rather than turning into a wall.

Avoid repeating the same turn direction forever by tracking the last turn and slightly biasing away from it if distances are close to equal.

Position the ultrasonic sensor thoughtfully mounted too low, it may miss low obstacles; mounted too high, it may miss table edges.



```
if (leftDistance < SAFE_DISTANCE && rightDistance < SAFE_DISTANCE) { moveBackward(150);
```

```
delay(400);

turnRight(150); // pick a consistent 'default' escape turn delay(500);

} else if (leftDistance > rightDistance) { turnLeft(150);

} else {

turnRight(150);

}
```

STUDENT CHALLENGE

Add a counter that tracks how many consecutive times the robot has turned the same direction, and force the opposite turn after 3 in a row.

Time how long your robot takes to cross a fixed-length corridor with 2 obstacles, and try to reduce that time by tuning SAFE_DISTANCE and turn duration.

Deliberately place the robot in a corner and document, step by step, what it does then fix any point where it gets stuck.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Obstacle-avoiding robots from Week 16; a test course with corners and dead-ends built from boxes.

Preparation checklist: Build one 'hard' test corner as a class obstacle course before this lesson.

Demonstration procedure: Run several teams' Week 16 robots into the same corner and compare failure modes as a class discussion before writing any fixes.

Questions to ask students: What real-world condition would make a robot's 'always turn right when stuck' rule fail? How would you test whether your fix actually worked, versus just seeming to work once?

Expected mistakes: Fixing one specific failure by hard-coding a reaction to that exact obstacle, rather than a general rule.

Assessment criteria: Robot successfully escapes a corner/dead-end test course without getting stuck in a repeating loop.

PROJECT 6 Automatic Floor-Mopping Robot (Week 18)

Problem the real-world problem we are solving

Manually mopping floors is repetitive and time-consuming. A robot that can move around a room, avoid obstacles, and dispense a controlled amount of water while dragging a cleaning pad can clean autonomously.

Learning objectives

- Reuse the 2WD platform and obstacle-avoidance logic without rebuilding them from scratch.

- Add a second actuator system (pump + reservoir) alongside movement.

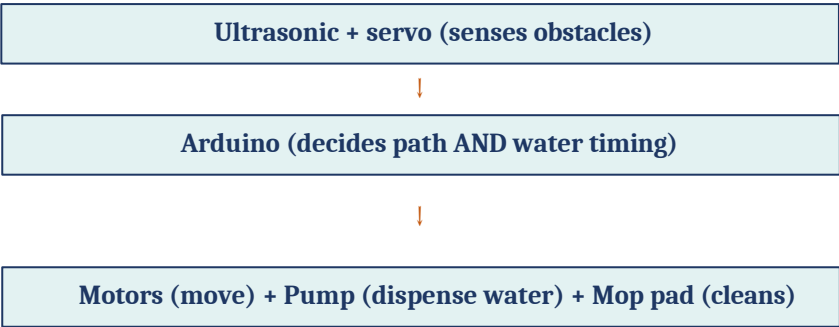
- Practise engineering integration: combining multiple previously separate subsystems into one coherent machine.

- Manage simple timing so water is dispensed steadily, not continuously.

Components (Bill of Materials)		

- 2WD chassis with motors, driver, ultrasonic + servo (from Project 3)
 - 1
 - Base autonomous platform
 - Small water reservoir
 - 1
 - Holds cleaning water
 - Mini pump
 - 1
 - Delivers water to the mop pad
 - Mop / cleaning pad mechanism
 - 1
 - Cleans the floor as the robot moves
 - Relay module (if pump needs separate switching)
 - 1
 - Controls the pump safely
 - Battery pack (sized for motors + pump)
 - 1
 - Power
 - IR sensors (optional)
 - Set
 - Extra edge/obstacle detection at floor level
- Power / safety note**
- The pump adds extra current draw on top of the drive motors recheck the battery pack's capacity against Level 10's power guidance before assuming the existing pack is sufficient.

System block diagram



How the system works

The robot reuses the exact obstacle-avoidance logic from Project 3 to move around the room and steer clear of obstacles. Running alongside that logic, on its own timer, the Arduino periodically pulses the pump on for a short, controlled burst to dampen the mop pad, then off again for longer so the floor gets a controlled amount of water rather than a constant stream.

Algorithm

Run the obstacle-avoidance algorithm from Project 3 continuously (move, sense, decide, turn).

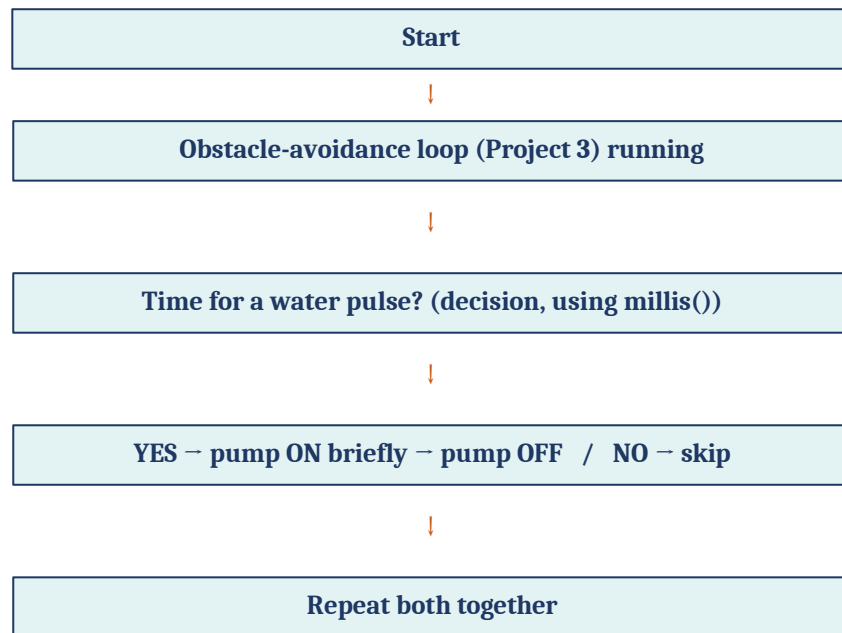
In parallel, track elapsed time since the pump last ran.

If enough time has passed, pulse the pump on briefly.

Turn the pump back off.

Continue both processes together, without one blocking the other.

Flowchart

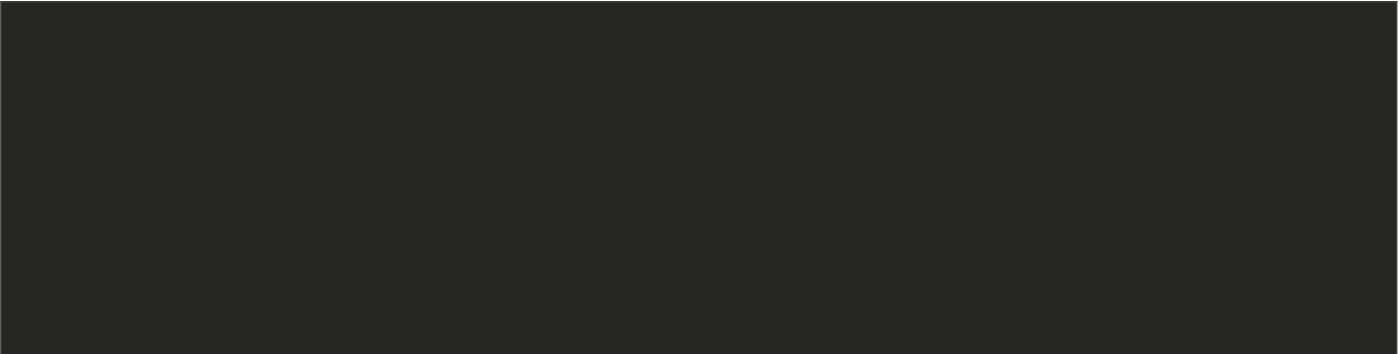


Build steps

Confirm the obstacle-avoidance robot from Project 3 still works exactly as before this is the foundation, unmodified.
Add the reservoir, pump, and mop pad mechanically, checking the added weight does not affect balance or motor load.
Test the pump alone (with the relay if used) before combining with movement.
Add timed pump pulses using `millis()`, never `delay()` (`delay()` would freeze the obstacle-avoidance sensing while pumping).
Test the combined system on a real (or towel-protected) floor at low speed first.

Version 1 Reused obstacle-avoidance only

Prove the base platform still works once the pump/reservoir are mechanically attached.



```
// Exactly the Project 3 final version, unchanged void loop() {  
  long distance = readDistanceCM(); if (distance > SAFE_DISTANCE) {  
    moveForward(150);  
  } else {  
    stopRobot();  
  }  
  // ...scan and turn logic from Project 3...  
}
```

Version 2 Add timed pump pulses using `millis()`

`millis()`-based timing runs 'in the background' without ever pausing the obstacle sensing this is why `delay()` is avoided here.



```

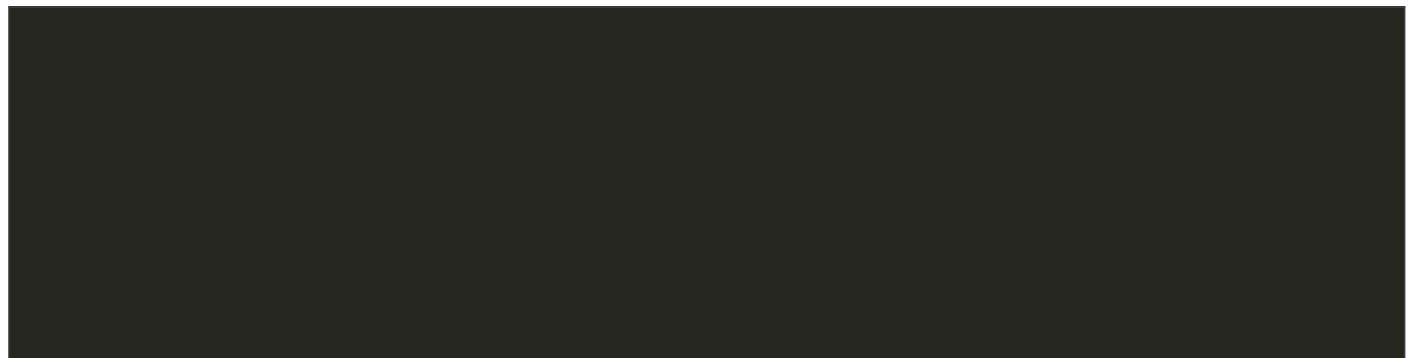
const int PUMP_PIN = 11; unsigned long lastPumpTime = 0;
const unsigned long PUMP_INTERVAL = 4000; // ms between pulses
const unsigned long PUMP_ON_TIME = 300;      // ms pump stays on each pulse
bool pumpIsOn = false;
unsigned long pumpStartTime = 0;

void handleMopping() {
  unsigned long now = millis();
  if (!pumpIsOn && now - lastPumpTime > PUMP_INTERVAL) { digitalWrite(PUMP_PIN, HIGH);
    pumpIsOn = true; pumpStartTime = now;
  }
  if (pumpIsOn && now - pumpStartTime > PUMP_ON_TIME) { digitalWrite(PUMP_PIN, LOW);
    pumpIsOn = false; lastPumpTime = now;
  }
}

```

Final Version Combine movement and mopping in one loop()

Both subsystems are checked every pass through loop() this is how a microcontroller does more than one 'thing at a time.'



```

void loop() {
  long distance = readDistanceCM(); if (distance > SAFE_DISTANCE) {
    moveForward(150);
  } else {
    stopRobot();
    // ...scan and turn logic from Project 3...
  }
  handleMopping(); // runs every loop, independent of movement state
}

```

Code explanation key lines

millis() returns the time since the program started; subtracting a stored 'last event' time and comparing it to an interval is the standard non-blocking timing pattern in embedded C++.

Using delay() for the pump would freeze obstacle sensing for that entire duration exactly the mistake this version avoids. handleMopping() is deliberately a separate function so movement logic and cleaning logic stay easy to read and debug independently.

Testing procedure

Test pump pulses alone (robot stationary) and confirm timing matches PUMP_INTERVAL and PUMP_ON_TIME.

Test obstacle avoidance alone with the pump disconnected.

Test both together and confirm the robot never pauses movement while pumping.

Test on an actual floor with a towel underneath to protect electronics from splashes during debugging.

Common problems

Robot pauses briefly every few seconds

delay() used somewhere in the pump logic instead of millis()

Water leaks near electronics

Reservoir/tubing not sealed or routed away from the driver board and Arduino

Pump never triggers

PUMP_INTERVAL check placed inside an if-block that isn't always reached

Debugging checklist

Is delay() used anywhere near the pump logic? If so, replace it with millis()-based timing.

Is handleMopping() called on every single pass through loop(), unconditionally?

Is the reservoir sealed and routed away from all electronics?

Does the added pump/reservoir weight require rebalancing the chassis?

STUDENT CHALLENGE

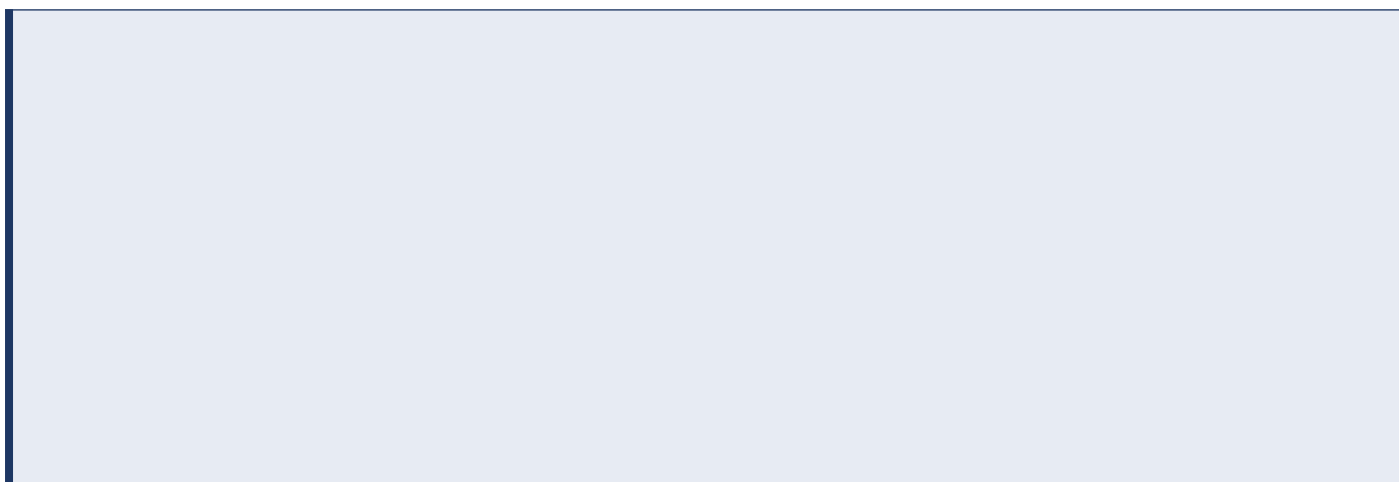
Add a low-water warning: track approximate pulses dispensed and flag when a set number is reached.

Make PUMP_INTERVAL depend on how fast the robot is moving (fresh water more often at higher speed). Add a manual pump-test button, separate from the automatic timer, for maintenance.

Extension ideas

Add a simple wiper/scrubber servo motion synchronised with pump pulses.

Add IR floor sensors to detect stairs/drops and stop before falling.
Track total floor area 'cleaned' using time and speed as a rough estimate.
Enter it in the Week 20 Robotics Challenge for best engineering integration.



GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Assembled obstacle-avoiding robots from Phase 4; small reservoirs/pumps; towels/trays to protect the test floor and electronics.

Preparation checklist: Test every pump and reservoir for leaks before class, away from any electronics.

Demonstration procedure: Show the delay()-based pump timing pausing the whole robot, then show the millis()-based fix running smoothly a very visible before/after.

Questions to ask students: Why can't delay() be used for the pump timing here, when it was fine for blinking an LED in Level 7? What changed?

Expected mistakes: Reservoir or tubing leaking onto the driver board; forgetting to re-test the base obstacle-avoidance behaviour after adding the extra mechanical weight.

Assessment criteria: Robot moves, avoids obstacles, and dispenses water on a steady timed cycle, all simultaneously, without pausing.

PROJECT 7 Soccer Robot (Week 19)

Problem the real-world problem we are solving

A robot that can navigate is not the same as a robot that can play a game: it must find a specific object (the ball), position itself, and take a deliberate action (kicking) at the right moment.

Learning objectives

- Use IR or ultrasonic sensors to search for and centre on a target.
- Break complex behaviour into small, named functions.
- Coordinate a kicking mechanism with positioning logic.
- Practise iterative testing of a multi-step autonomous behaviour.

Components (Bill of Materials)

2WD or 4WD chassis with motors and driver

1

Robot platform

IR sensors or ball-detection sensors (2-3)

Set

Detect the ball's direction

Ultrasonic sensor

1

Detect the goal/obstacles

Servo motor

1

Powers the kicking mechanism

Simple kicking mechanism (lever/arm)

1

Strikes the ball on command

Battery pack

1

Power

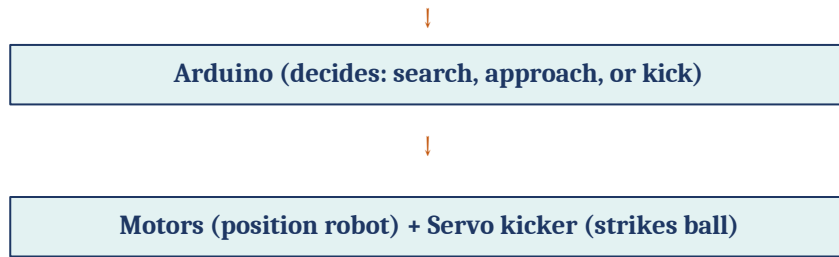
Remote-control module (optional)

1

Manual override during testing/matches

System block diagram

IR sensors (find ball) + Ultrasonic (find goal/obstacles)



How the system works

The robot continuously checks its IR sensors to determine whether the ball is to its left, right, or directly ahead. If no ball is detected, it searches by turning slowly. Once the ball is roughly centred and close, the robot stops and triggers the servo-powered kicking mechanism. Breaking this into small functions `searchForBall()`, `moveForward()`, `turnLeft()`, `turnRight()`, `kickBall()`, `stopRobot()` keeps the overall logic in `main()/loop()` short and readable, matching how professional robotics code is structured.

Algorithm

Read the ball-detection sensors.

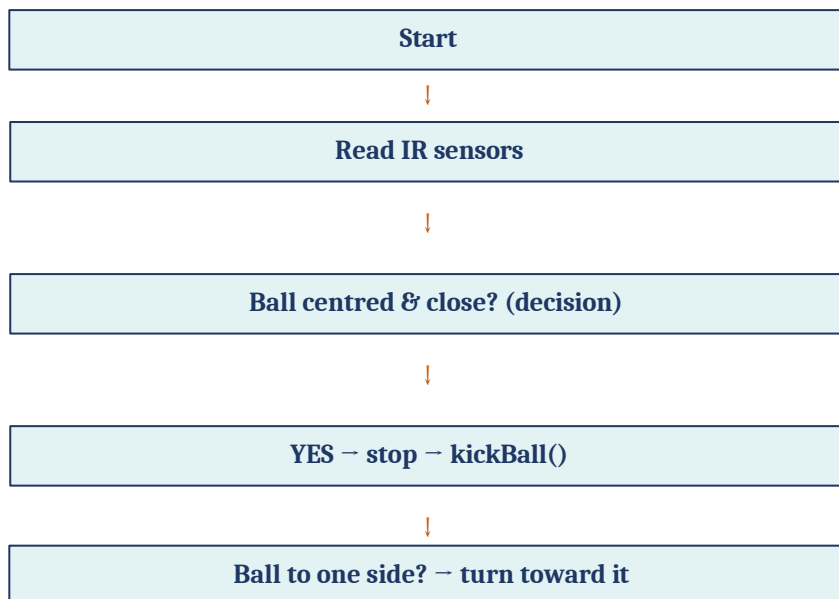
Is the ball detected ahead and close? If yes, stop and kick.

Is the ball detected to one side? If yes, turn toward it.

If the ball is not detected at all, search by turning slowly.

Repeat continuously.

Flowchart



↓

No ball found → searchForBall() (turn slowly)

↓

Repeat

Build steps

Test each IR sensor alone, confirming it correctly reports 'ball detected/not detected' at a realistic distance.

Write and test searchForBall() alone (robot should turn slowly and repeatedly when no ball is present).

Add ball-centring logic using the movement functions from Phase 4, unmodified.

Build and test the kicking mechanism alone with the servo, before linking it to detection.

Combine detection, positioning, and kicking into the full behaviour.

Test repeatedly with the ball placed in different starting positions.

Version 1 Search only (no ball yet)

Confirms the search behaviour before any detection logic exists.

```
void searchForBall() {  
  
  turnRight(120); // slow search turn  
  
}  
  
void loop() {  
  searchForBall();  
}
```

Version 2 Add ball detection and centring

```

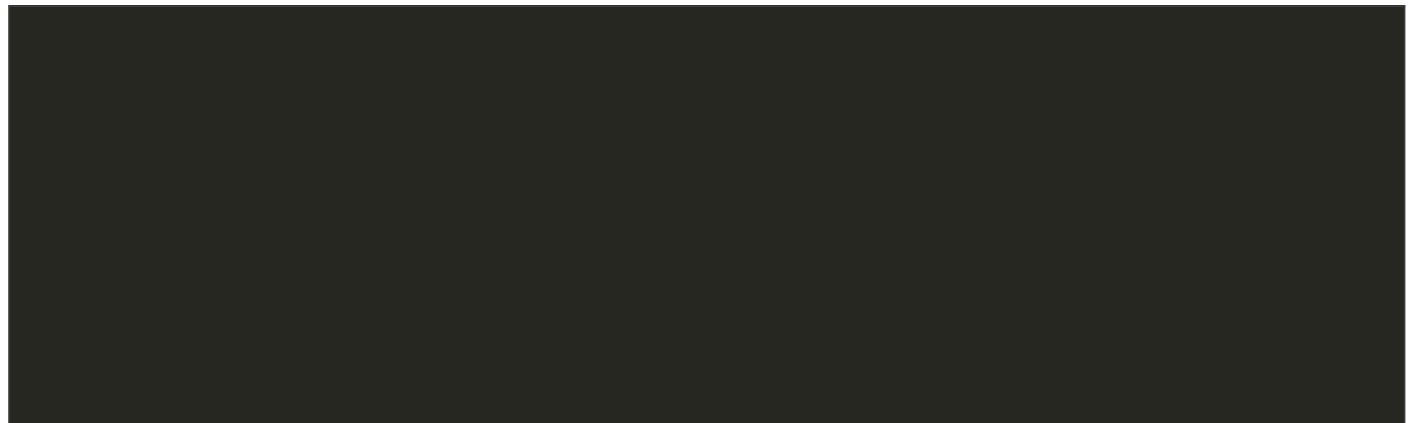
bool ballLeft()    { return digitalRead(IR_LEFT_PIN) == LOW; } bool ballRight() { return
digitalRead(IR_RIGHT_PIN) == LOW; } bool ballCenter() { return digitalRead(IR_CENTER_PIN) ==
LOW; }

void loop() {
  if (ballCenter()) { moveForward(150);
  } else if (ballLeft()) { turnLeft(120);
  } else if (ballRight()) { turnRight(120);
  } else {
    searchForBall();
  }
}

```

Final Version Add distance check and kicking

Full behaviour: search, approach, position, kick.



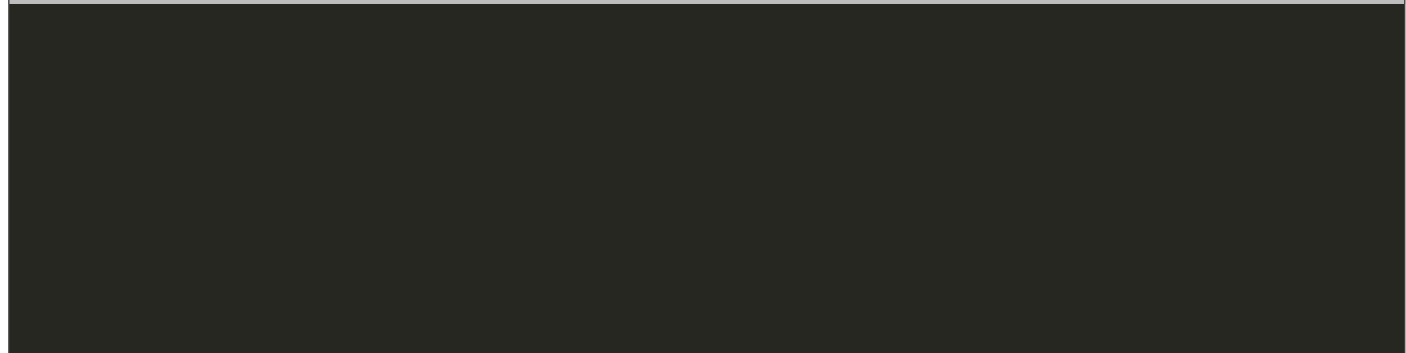
```

void kickBall() { kickerServo.write(120); // strike delay(300);
kickerServo.write(0);    // reset
}

void loop() {
  long distance = readDistanceCM();

  if (ballCenter() && distance < 10) { stopRobot();
  kickBall();
  }
}

```



```
} else if (ballCenter()) { moveForward(150);  
}  
} else if (ballLeft()) { turnLeft(120);  
}  
} else if (ballRight()) { turnRight(120);  
}  
} else {  
  searchForBall();  
}  
}  
}
```

Code explanation *key lines*

ballLeft(), ballRight(), and ballCenter() are small functions that each answer one true/false question this makes the main loop() read almost like a description of soccer strategy in English.

The distance < 10 condition prevents the robot from 'kicking' at empty air the moment the ball is merely centred but still far away.

kickBall() resets the servo back to a starting position after each strike so it is ready to kick again.

Testing procedure

Test each IR sensor's detection range and adjust placement/sensitivity before writing any movement logic.

Test searchForBall() alone in an empty area the robot should rotate steadily, not erratically.

Test the kicking mechanism alone, triggered manually, to confirm it strikes with enough force.

Test the full sequence with the ball placed left, right, and directly ahead in separate trials.

Common problems

Robot ignores the ball

IR sensors placed too high/low or too sensitive/insensitive for the ball's surface

Robot kicks at empty air

Distance threshold too large, or missing distance check entirely

Robot spins continuously without settling

Left/right conditions overlapping so neither is ever fully satisfied

Debugging checklist

- Does each ball-detection function work correctly on its own, tested with the ball moved by hand?
- Is there a genuine distance check before kicking, not just a 'ball centred' check?
- Are the search, approach, and kick behaviours each individually testable and tested?
- Is the kicking mechanism resetting properly after every kick?

STUDENT CHALLENGE

Add a 'possession timer': if the ball is lost from view for more than 2 seconds after being centred, resume searchForBall(). Tune the search turn speed to find the ball faster without overshooting past it.

Add an ultrasonic-based simple 'avoid the goalpost' rule while searching.

Extension ideas

- Add a second robot and a small goal for real practice matches.
- Add the Bluetooth remote control from Project 2 as a manual override for referees/coaches.
- Track and print a simple 'kicks attempted' counter for match statistics.
- Enter the Week 20 Robotics Challenge soccer competition.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: 2WD/4WD chassis with IR sensors and a simple kicking mechanism per group; a lightweight ball; a small marked 'pitch' area.

Preparation checklist: Test IR sensor detection range against the actual ball to be used sensitivity varies a great deal by ball colour and material.

Demonstration procedure: Demonstrate each of the six named functions in isolation on the whiteboard/terminal before

showing how they compose into the full loop() reinforcing why functions make complex behaviour readable. Questions to ask students: Why is the code organised into six small functions instead of one long loop()? What real soccer skill does each function represent?

Expected mistakes: Writing the entire behaviour as one long block of nested if-statements instead of named functions; testing only with the ball dead-centre and never off to a side.

Assessment criteria: Robot reliably finds, approaches, and kicks a ball placed in at least three different starting positions.

Robotics Challenge (Week 20 Phase Milestone)

Real-life problem

Engineers rarely get a working system right the first time. A competitive, public test of their robots where something invariably goes wrong live is one of the fastest ways students learn to debug their own systems rather than depend on the for corrected code.

Suggested competition categories

Fastest obstacle avoidance

Time to cross a course with obstacles without collision

Best autonomous navigation

Fewest collisions/stalls over a fixed, harder course

Soccer robot competition

Goals scored in a short timed match

Most reliable robot

Fewest failures across repeated runs

Best engineering design

Judged on build quality, wiring neatness, and code organisation

role during the challenge

Do not fix a team's robot for them during the challenge. Ask diagnostic questions instead: 'What exactly is it doing wrong?' 'Which sensor reading would tell you why?' Let students apply the Debugging Guide under real time pressure this is the point of the exercise.

STUDENT CHALLENGE

Before the challenge, write down (on paper) your robot's three most likely failure points, based on everything you've learned so far.

During practice runs, keep a short log: what failed, what you changed, whether it worked afterward.

After the challenge, write two sentences on what you would redesign if you built this robot again from scratch.

GUIDE

Suggested time: 4 hours (2 classes in a week) Milestone competition

Equipment needed: All robots from Phase 4-5; a marked course with obstacles; a small goal area for soccer robots; stopwatch/timer.

Preparation checklist: Set up the course the day before; test it yourself with a spare robot to confirm it is fair and achievable.

Demonstration procedure: None this week is entirely student-led practice and competition.

Questions to ask students: What did your robot do differently between practice and the real run, and why? Expected mistakes: Panicking and randomly changing code without first identifying what specifically is failing.

Assessment criteria: MILESTONE: every team competes with a functioning robot in at least one category and can explain, afterward, one specific thing they debugged themselves during the day.

PHASE 6 ESP32 & Smart Systems (Weeks 21-22, 8 hours)

Your robot can now connect to the internet. Welcome to the Internet of Things (IoT). Your projects are no longer limited to what is physically nearby.

Every project so far has run on an Arduino Uno, communicating only with what is physically wired to it. This phase introduces the ESP32 a microcontroller with built-in Wi-Fi and Bluetooth completing the picture of what modern embedded systems and IoT (Internet of Things) can do.

LEVEL ESP32 Fundamentals (Week 21)

Real-life problem

A farmer travelling away from their field still wants to know if the soil is dry. An Arduino Uno cannot tell them remotely it has no way to connect to the internet on its own. The ESP32 solves exactly this problem.

What makes ESP32 different from Arduino Uno

Built-in Wi-Fi

No

Yes

Built-in Bluetooth/BLE

No

Yes

Clock speed

16 MHz

~240 MHz (dual-core)

Operating voltage

5V logic

3.3V logic (check sensor compatibility!)

Typical use in this course

Learning fundamentals, simple robots

IoT projects, smart home, remote monitoring

Important: 3.3V logic

ESP32 pins operate at 3.3V, not 5V like the Uno. Connecting a 5V sensor's signal pin directly to an ESP32 GPIO pin can damage it. Always check a sensor/module's voltage compatibility, and use a level shifter or voltage divider where needed.

ESP32 GPIO, analog input, and PWM

digitalRead(), digitalWrite(), analogRead(), and the general PWM concept from Level 9 all work the same way conceptually on ESP32

the exact pin numbers and some capabilities (e.g. which pins support analogRead()) differ from the Uno and should be checked against the specific ESP32 board's pinout diagram.

Connecting to Wi-Fi

```
#include <WiFi.h>

const char* ssid = "YourWiFiName";
const char* password = "YourWiFiPassword";

void setup() { Serial.begin(115200); WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) { delay(500);
Serial.print(".");
```

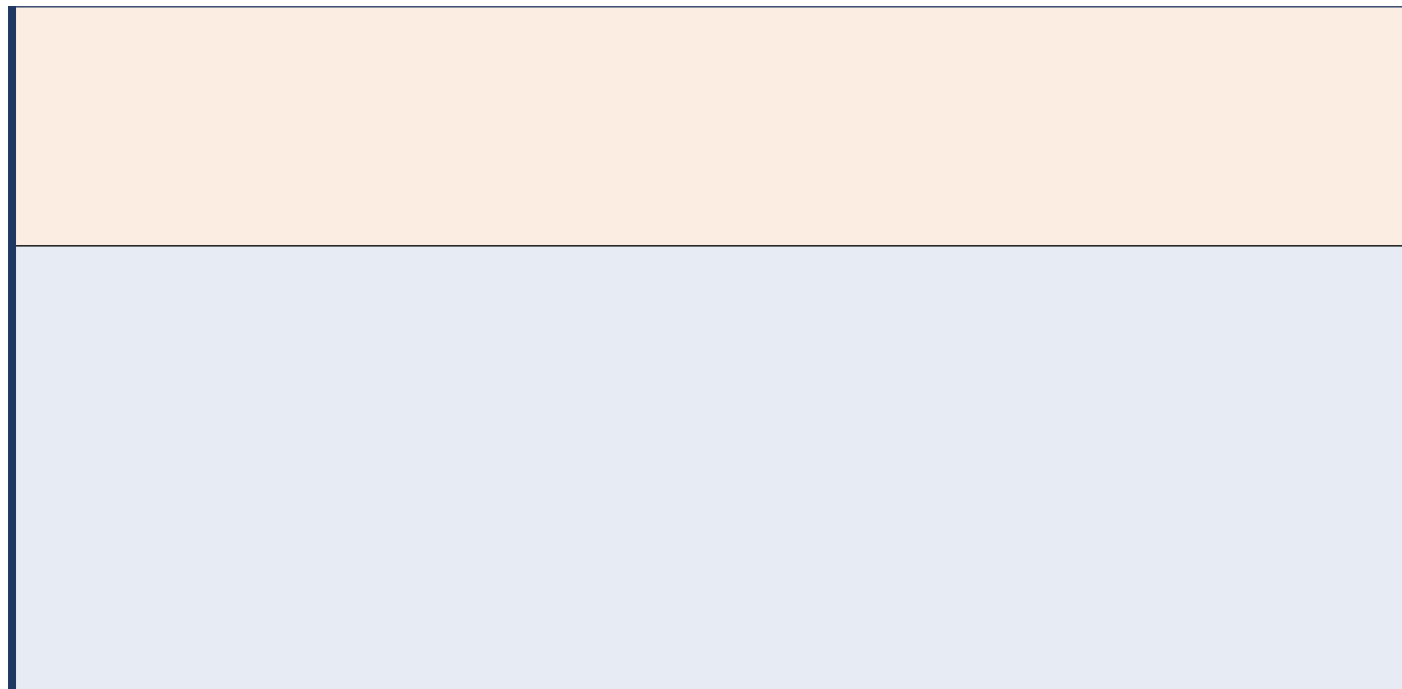
```
}  
  
Serial.println("Connected!"); Serial.println(WiFi.localIP());  
  
}  
  
  
  
void loop() {  
  
}
```

Why did we write it this way?

The while loop deliberately blocks here because connecting to Wi-Fi is a one-time setup step nothing useful can happen until it succeeds, unlike sensor reading, which must never block using `delay()` for long periods.

Bluetooth/BLE on ESP32

ESP32's built-in Bluetooth means projects like the Project 2 remote-controlled car could be rebuilt without a separate HC-05 module the ESP32 itself can act as a Bluetooth Serial device.



STUDENT CHALLENGE

Connect your ESP32 to the school/home Wi-Fi and print its assigned IP address to the Serial Monitor.

Compare an ESP32 pinout diagram to the Arduino Uno's and list three concrete differences you would need to account for when moving a project across.

Read an analog sensor (e.g. the potentiometer from Level 8) on the ESP32 and confirm the value range is it still 0-1023, or different?

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: ESP32 dev boards (one per group); classroom Wi-Fi credentials shared securely.

Preparation checklist: Confirm ESP32 board support is installed in every Arduino IDE beforehand, and pre-test Wi-Fi connectivity in the actual classroom.

Demonstration procedure: Connect an ESP32 live and open the Serial Monitor to show the assigned IP address make Wi-Fi connectivity feel concrete, not abstract.

Questions to ask students: Why must sensor voltage compatibility be checked before wiring to an ESP32, when it wasn't a concern on the Uno? What does 'blocking' code mean, and why is it acceptable in Wi-Fi setup but not in a sensor loop?

Expected mistakes: Wiring a 5V sensor output directly into an ESP32 pin; assuming Uno pin numbers carry over unchanged.

Assessment criteria: Student successfully connects their ESP32 to Wi-Fi and reads at least one sensor correctly adapted for

3.3V logic.

PROJECT 8 Smart Home / Security System (Week 21-22)

Problem the real-world problem we are solving

A home left empty needs a way to detect intruders, light itself automatically, and alert its owner all without a human physically monitoring it around the clock.

Learning objectives

- Combine multiple sensors into one coordinated system.
- Implement a manual arm/disarm override alongside automatic behaviour.
- Display system status clearly using an OLED/LCD.
- Practise integrating several previously separate SENSE–THINK–ACT chains into one program.

Components (Bill of Materials)

ESP32
1
Main controller
PIR motion sensor
1
Detects movement
LDR
1
Detects ambient light for automatic lighting
Buzzer
1
Sounds the alarm
LEDs (2+)
Set

Automatic lighting + status indicator

Relay module

1

Switches a lamp or lock mechanism

OLED/LCD display

1

Shows system status (armed/disarmed, alerts)

Push button

1

Manual arm/disarm

Door sensor (magnetic reed switch, optional)

1

Detects door opening

System block diagram

PIR + LDR + door sensor (sense)



ESP32 (decides: alarm, light, or ignore)



Buzzer + LEDs + relay (act) + OLED (status)

How the system works

The system has two independent behaviours running together. Automatic lighting: whenever it is dark (LDR) and motion is detected (PIR), a light turns on regardless of arm state, like a normal porch light. Security alarm: only while the system is armed (toggled by the push button), motion or an opened door immediately sounds the buzzer and lights a warning LED. The OLED continuously shows whether the system is armed or disarmed and the most recent event.

Algorithm

Read the arm/disarm button and toggle system state if pressed.

Read the PIR, LDR, and door sensor.

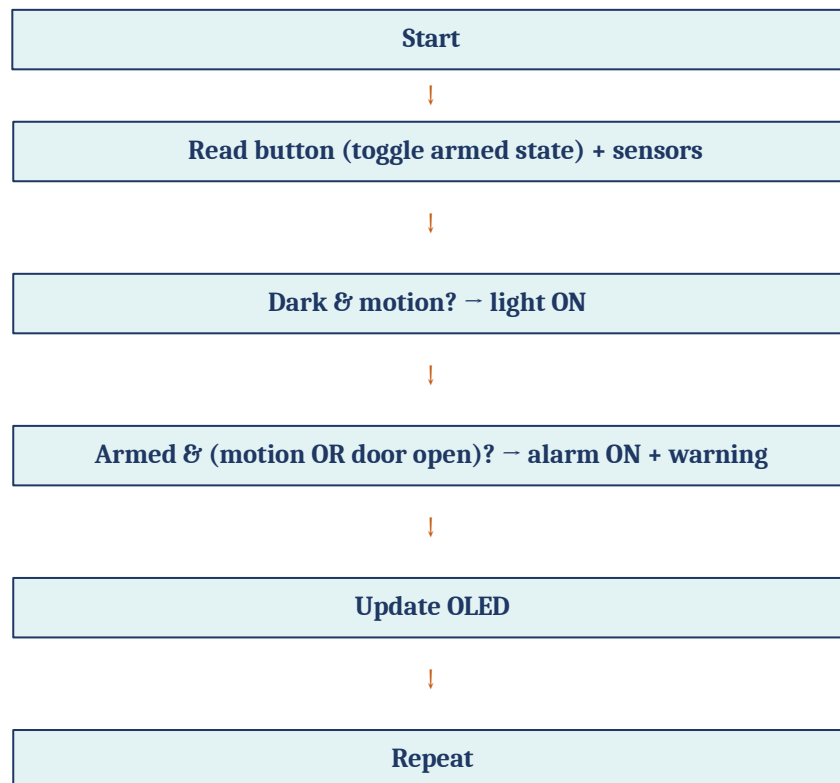
If it is dark and motion is detected: turn on the light (always, regardless of arm state).

If the system is armed and (motion is detected OR the door is opened): sound the alarm and show a warning.

Update the OLED display with current status.

Repeat continuously.

Flowchart



Build steps

Test the PIR, LDR, and door sensor individually, exactly as taught in Level 8, before combining anything.

Test the buzzer, LEDs, and relay individually, as in Level 9.

Add the OLED and confirm it can display simple static text.

Add the automatic lighting behaviour alone and test it.

Add the arm/disarm button and armed-state alarm behaviour alone, then combine both behaviours.

Test edge cases: motion while disarmed, door opened while armed, toggling arm state during an active alarm.

Version 1 Automatic lighting only

The simplest SENSE–THINK–ACT chain in this project, built first.

```
const int PIR_PIN = 4,  
const int LDR_PIN = 34; // ESP32 ADC-capable pin const int LIGHT_PIN = 5;  
const int DARK_THRESHOLD = 1500; // calibrate for ESP32's ADC range
```

```
void loop() {  
  bool motion = digitalRead(PIR_PIN) == HIGH;
```

```
  int light = analogRead(LDR_PIN);
```

```
  if (light < DARK_THRESHOLD && motion) { digitalWrite(LIGHT_PIN, HIGH);  
  } else {  
    digitalWrite(LIGHT_PIN, LOW);  
  }  
}
```

Version 2 Add arm/disarm button and alarm



```
const int BUTTON_PIN = 15; const int BUZZER_PIN = 18; const int DOOR_PIN = 19; bool armed = false;

bool lastButtonState = HIGH;

void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);
  if (buttonState == LOW && lastButtonState == HIGH) { armed = !armed; // toggle on press
  }
  lastButtonState = buttonState;

  bool motion = digitalRead(PIR_PIN) == HIGH; bool doorOpen = digitalRead(DOOR_PIN) == HIGH;

  if (armed && (motion || doorOpen)) { digitalWrite(BUZZER_PIN, HIGH);
  } else {
    digitalWrite(BUZZER_PIN, LOW);
  }
}
```

Final Version Combine lighting, alarm, and OLED status

Both SENSE–THINK–ACT chains run in the same loop(), each checked every pass.

```
void loop() {  
  
  handleArmToggle(); // button logic handleAutoLighting(); // PIR + LDR + light handleAlarm();  
    // armed + PIR/door + buzzer updateDisplay();          // OLED status  
  
}
```

Code explanation key lines

Comparing buttonState to lastButtonState detects the exact moment the button changes from not-pressed to pressed, so holding it down doesn't toggle armed on and off rapidly.

The lighting and alarm behaviours are deliberately kept independent (lighting works whether armed or not) this matches how a real porch light behaves.

Splitting the final version into four named functions (handleArmToggle, handleAutoLighting, handleAlarm, updateDisplay) keeps a fairly complex system's loop() readable at a glance.

Testing procedure

1. Test automatic lighting alone in a darkened room with hand-waving to trigger the PIR.
2. Test the arm/disarm button alone, confirming a single press toggles state cleanly.
3. Test the alarm while armed, and confirm it stays silent while disarmed under the same motion.
4. Test the OLED updates correctly reflect every state change.

Common problems

Button toggles multiple times per press

Missing edge-detection (comparing current vs last state) a 'debounce' problem

Alarm sounds even when disarmed

armed variable not actually being checked in the alarm condition

Light works but alarm doesn't (or vice versa)

One handle function not being called every loop(), or an early return skipping it

Debugging checklist

1. Does each sensor work correctly in isolation before being combined?
2. Is the button toggle based on a state change, not just 'is it currently pressed'?
3. Are all four handler functions actually called on every pass through loop(), with nothing skipping them?
4. Does the OLED status match reality at every stage of manual testing?

STUDENT CHALLENGE

Add a short delay/grace period after arming before the alarm becomes active, so the owner can leave the house. Add an LED colour or OLED message that distinguishes 'motion alarm' from 'door alarm.'

Log the time (using millis()) since startup) of the most recent alarm event and show it on the OLED.

Extension ideas

5. Send a Wi-Fi notification (e.g. to a simple webpage or messaging service) when the alarm triggers.
6. Add a keypad for a PIN-based arm/disarm instead of a single button.
7. Add a camera module (if available) to capture an image when the alarm triggers.
8. Combine with the relay to actually lock/unlock a small solenoid door lock.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: ESP32 + PIR + LDR + buzzer + LEDs + relay + OLED + button + door sensor per group. Preparation checklist: Test PIR sensors' warm-up time (many need ~30-60 seconds after power-on to stabilise) and warn students so they don't mistake this for a fault.

Demonstration procedure: Demonstrate the two behaviours (auto-light and alarm) completely separately first, then show them combined reinforces the 'independent SENSE→THINK→ACT chains in one loop()' concept.

Questions to ask students: Why should the automatic light work regardless of the armed state? What real security-system feature would prevent a false alarm from someone arming the system and then walking to the door themselves?

Expected mistakes: Confusing armed toggle logic with a simple 'is button pressed' check, causing rapid on/off toggling; PIR false triggers during its warm-up period mistaken for a wiring fault.

Assessment criteria: System correctly and independently manages automatic lighting and an armed-state alarm, with accurate OLED status throughout.

PROJECT ESP32 IoT Smart Farm / Smart Home Monitoring System (Week 22)

Problem the real-world problem we are solving

A farmer or homeowner wants to check environmental conditions (soil moisture, temperature, humidity) without physically being present. Sensor data needs to leave the device and become viewable remotely.

Learning objectives

- 9. Combine multiple sensors on one ESP32.
- 10. Understand, at a beginner level, how sensor data can be transmitted over Wi-Fi.
- 11. Display live readings locally on an OLED as well as remotely.
- 12. Reuse sensor code (DHT22, soil moisture) built earlier in the course almost unchanged.

Components (Bill of Materials)

ESP32

1

Main controller with Wi-Fi

Capacitive soil moisture sensor

1

Soil monitoring (reused from Project 4)

DHT22 temperature/humidity sensor

1

Air condition monitoring

OLED display

1

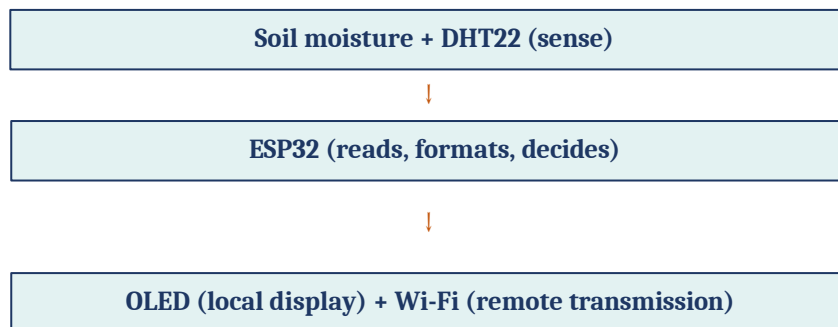
Local live readout

Breadboard + jumper wires

Set

Assembly

System block diagram



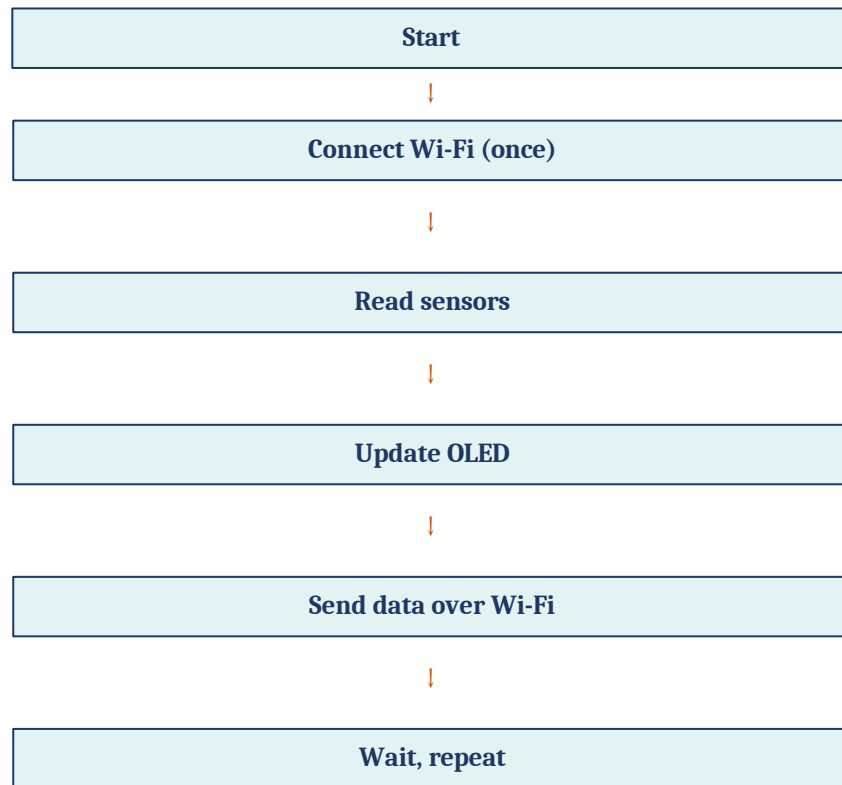
How the system works

The ESP32 reads soil moisture and air temperature/humidity at a regular interval, exactly as in Project 4 and the DHT sensor reference in Level 8. It displays the current readings on a local OLED for anyone standing at the device, and because it is connected to Wi-Fi the same readings can be sent to a simple web page or a basic HTTP endpoint so the information can be checked remotely, completing the 'IoT' (Internet of Things) picture: a physical sensor's data becoming accessible over the internet.

Algorithm

1. Connect to Wi-Fi once, at startup.
2. Read soil moisture and temperature/humidity.
3. Display the readings on the OLED.
4. Send the readings over Wi-Fi (to a simple server/web page).
5. Wait a sensible interval.
6. Repeat from step 2.

Flowchart



Build steps

1. Confirm Wi-Fi connects reliably (Week 21).
2. Wire and test soil moisture and DHT22 sensors individually, reusing Project 4 and Level 8 code directly.
3. Add the OLED and display both readings, formatted clearly.
4. Add a simple Wi-Fi data-sending step (this can be as simple as printing a URL-style string to Serial for a class discussion, or a real HTTP GET request for more advanced groups).
5. Test the complete cycle end-to-end at a sensible reading interval (e.g. every 10-30 seconds, not continuously).

Version 1 Read and display sensors locally

No networking yet reuses Project 4 and Level 8 sensor code directly.

```
#include <DHT.h> #define DHTPIN 4 #define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);
const int SOIL_PIN = 34;

void setup() { Serial.begin(115200); dht.begin();
}

void loop() {
float temp = dht.readTemperature(); float hum = dht.readHumidity();
int soil = analogRead(SOIL_PIN);

Serial.print("Temp: "); Serial.print(temp); Serial.print("C Humidity: ");
Serial.print(hum); Serial.print("% Soil: "); Serial.println(soil); delay(10000);
}
```

Version 2 Add Wi-Fi connection

Reuses the Week 21 Wi-Fi connection code unchanged.

```
#include <WiFi.h>
const char* ssid = "YourWiFiName";
```

```
const char* password = "YourWiFiPassword";

void setup() { Serial.begin(115200); dht.begin(); WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) { delay(500);
}
Serial.println(WiFi.localIP());
}
```

Final Version Send readings over Wi-Fi

A simple HTTP GET request to a basic logging endpoint the exact endpoint depends on the class's chosen simple dashboard/service, discussed by the .

```
#include <HTTPClient.h>

void sendReading(float temp, float hum, int soil) { if (WiFi.status() == WL_CONNECTED)
{
HTTPClient http;
String url = "http://your-server-address/update?t=" + String(temp)X
+ "&h=" + String(hum) + "&s=" + String(soil); http.begin(url);
int code = http.GET(); Serial.println(code); http.end();
}
}
```

Code explanation key lines

1. The DHT22 and soil moisture code is deliberately identical in structure to what students already built in earlier weeks the new skill this week is transmission, not sensing.
2. `WiFi.status() == WL_CONNECTED` is checked before every send attempt so the device does not crash trying to use a dropped connection.
3. `String(temp)` converts a numeric reading into text so it can be embedded into a web address/URL.

Testing procedure

1. Test sensors and OLED locally before adding any networking.
2. Test the Wi-Fi connection alone, confirming a stable IP address.
3. Test one manual `sendReading()` call and confirm the server/endpoint receives it.
4. Test the full 10-30 second cycle running for several minutes without crashing or disconnecting.

Common problems

Readings show as NaN

DHT wiring/pull-up resistor issue, or reading too frequently

Wi-Fi keeps disconnecting

Weak signal, wrong password, or ESP32 power supply insufficient during Wi-Fi transmission bursts

Server never receives data

Wrong IP/URL, firewall blocking the request, or `WiFi.status()` check silently skipping every send

Debugging checklist

1. Do the sensors work correctly with no networking code present at all?
2. Does `WiFi.localIP()` print a sensible address on the classroom network?
3. Is the exact URL/endpoint being printed to Serial for every attempt, so it can be checked by eye?
4. Is the reading interval sensible (not so frequent it floods the network or sensor)?

STUDENT CHALLENGE

Add the reading interval as a named constant (`READING_INTERVAL`) instead of a hard-coded number. Display the Wi-Fi connection status (connected/disconnected) on the OLED at all times.

Add a simple minimum/maximum tracker for temperature over the session and display it.

Extension ideas

5. Build a very simple webpage that displays the latest reading (a natural next step for stronger groups, taught as an extension rather than core curriculum).
6. Add more sensors (e.g. light, water level) to the same monitoring system.
7. Add a low-power 'deep sleep' mode between readings to extend battery life introduces students to real-world IoT power constraints.
8. Combine with Project 4's irrigation logic so watering decisions can also be checked/logged remotely.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: ESP32 + soil sensor + DHT22 + OLED per group; reliable classroom Wi-Fi; a simple server/endpoint or dashboard prepared in advance by the teacher for the class to send data to.

Preparation checklist: Set up one simple shared logging endpoint or dashboard in advance (this can be as simple as a spreadsheet-backed web form) so all groups have somewhere real to send data.

Demonstration procedure: Show one working reading appearing on the shared dashboard live, then walk back through the three code versions that built up to it.

Questions to ask students: What is the difference between the sensor reading itself and 'sending' it where does each piece of code belong? Why check `WiFi.status()` before every send instead of assuming it's still connected?

Expected mistakes: Sending data far too frequently, overwhelming the shared class endpoint; forgetting that DHT sensors need a minimum time between reads.

Assessment criteria: Student's ESP32 correctly reads two sensors, displays them locally, and successfully transmits at least one reading to the shared class endpoint.

PHASE 7 Capstone Engineering Project (Weeks 23-24, 8 hours)

You design and build your own project. No template. No step-by-step guide. Just you, your team, and a problem you want to solve.

This is the course's final proof of learning. Students no longer follow a provided project brief they choose a real-world problem, design their own system, and build it using everything from Phases 1-6. The capstone is done in teams of 2-4.

The Brief

Each team chooses ONE problem to solve using an embedded/robotics system built around SENSE – THINK – ACT. Suggested problem areas (teams may also propose their own, subject to approval and hardware availability):

9. Smart irrigation for a specific crop/garden bed
10. Automatic feeding system for a specific animal
11. A cleaning robot for a specific surface/space
12. A soccer robot with an improved feature over Project 7
13. A smart security system for a specific space
14. A smart home comfort system (lighting, ventilation, temperature)
15. A smart farm monitoring dashboard
16. A small delivery/transport robot
17. An original ESP32 IoT system

Required Submission (10 parts)

- Problem statement what real-world problem, for whom, and why it matters.
- System design a short written description of the overall approach (the SENSE – THINK – ACT chain(s) involved).
- Component list a full Bill of Materials in the same table format used throughout this manual.
- Circuit diagram clearly labelled (VCC/GND/signal/analog/digital/motor/external power, as taught in Section 10 of this manual).
- Flowchart of the system's main decision logic.
- Algorithm written in plain numbered steps, before any code.
- C++ code built in versions (V1 – V2 – V3 – Final), exactly as every project in this manual was built.
- Working prototype an actual physical build, not a simulation only.
- Testing results what was tested, what failed, what was fixed, and how the team knew it was fixed.
- Short presentation 5-8 minutes, demonstrating the system live and explaining the code to the class.

Week 23 Design & Build

Real-life problem

Every project in this course, up to now, arrived with a ready-made problem statement and BOM. Real engineering starts before any of that exists with a team deciding, together, what problem is actually worth solving and what it will take to solve it.

Suggested session flow

- Team selects and writes a one-paragraph problem statement; approves scope and hardware feasibility.
- Team drafts their system design, BOM, circuit diagram, flowchart, and algorithm all on paper/document, before any code.
- Team builds Version 1: the smallest possible working piece of hardware + code (e.g. one sensor reading correctly).
- Team incrementally adds features, testing after every addition, exactly as every project in this manual has modelled.
- Team documents each testing round as they go, not retroactively at the end.

STUDENT CHALLENGE

Before building anything, list the three riskiest/least-certain parts of your design the parts most likely to fail. Build and test those first.

Write your algorithm in plain numbered steps and have another team read it without seeing your code can they predict what your system will do?

Identify which previous project in this manual is closest to yours, and list exactly what code/knowledge you can reuse versus what is genuinely new.

GUIDE

Suggested time: 4 hours (2 classes)

Equipment needed: Full classroom hardware inventory available for teams to draw from, based on their approved BOM.
Preparation checklist: Review and approve every team's problem statement and BOM before building starts, checking hardware availability and realistic scope for the time remaining.

Demonstration procedure: None this is team-led design and build time. The teacher circulates, asking diagnostic questions rather than supplying answers.

Questions to ask students: What is the very first, smallest version of your system you could build today? What happens to your design if [a specific component] fails or isn't available?

Expected mistakes: Designing a system far too ambitious for two weeks; skipping the paper design step and going straight to wiring; choosing a problem with no clear SENSE – THINK – ACT structure.

Assessment criteria: Team has an approved problem statement, complete design documentation, and a working Version 1 by the end of the week.

Week 24 Testing, Debugging & Demonstration

Real-life problem

A system that has never been deliberately tested against failure is not finished it is untested. Professional engineers spend as much time breaking and fixing their own systems as they spent building them the first time.

Suggested session flow

- Team runs their system through a full test plan, deliberately trying edge cases (e.g. what happens in total darkness, or with the sensor covered).
- Team uses the Debugging Guide (see Appendix) to diagnose and fix at least one real problem, documenting the process.
- Team finalises their presentation: live demonstration plus a walk-through of their Final code version.
- Class demonstrations: each team presents for 5-8 minutes.
- Peer and assessment using the rubric below.

Capstone Assessment Rubric

Problem & design (20%)

Clarity of the problem statement; soundness of the SENSE–THINK–ACT design

Documentation (20%)

Complete BOM, circuit diagram, flowchart, algorithm, and version history

Working prototype (30%)

The system physically works and reliably performs its intended function

Code quality (15%)

Readable, organised into functions, sensibly named constants and variables

Testing & debugging (10%)

Evidence of deliberate testing and at least one real documented fix

Presentation (5%)

Clear explanation of the system and the code to the class

STUDENT CHALLENGE

During testing, find one condition under which your system currently fails, and document exactly how you found it.

Explain, in your presentation, one specific design decision you changed after testing revealed a problem not just what you built, but what you learned.

If you had two more weeks, what is the single most valuable improvement you would make next?

GUIDE

Suggested time: 4 hours (2 classes in a week) Final assessment

Equipment needed: All team hardware; a presentation space where each system can be demonstrated live.

Preparation checklist: Prepare the assessment rubric sheets in advance; schedule presentation order and timing so every team gets a fair, equal slot.

Demonstration procedure: None final student-led testing, debugging, and presentations.

Questions to ask students: What exactly did you test, and how do you know your fix actually solved the problem rather than just seeming to?

Expected mistakes: Presenting a system that has never been tested outside ideal conditions; being unable to explain a section of their own code.

Assessment criteria: FINAL MILESTONE: team delivers a working, demonstrated prototype with complete documentation and can explain every part of their own code and design decisions.

Appendix B Weekly Class Structure & Project Development Rule

The Standard 2-Hour Class Structure

- Recap
- 15 min
- Review the previous lesson's key idea before introducing anything new.
- New concept
- 25 min
- Introduce one major programming/electronics concept using the Real-Life Problem → Think → Concept pattern.
- Guided coding
- 25 min
- demonstrates while students write the code themselves, line by line.
- Practical work
- 40 min
- Students connect hardware and test their programs.
- Debugging
- 10 min
- Discuss common errors from the session and how to diagnose them.
- Challenge
- 5 min
- Set a modification/problem for students to attempt before the next class.

This structure is flexible during major project weeks, where practical work and debugging may reasonably occupy most or all of the session.

The Project Development Rule

Never hand over the finished project on day one.

Every major project in this manual is presented in this manual as Version 1 → Version 2 → Version 3 → Final. This is not a writing convenience it is exactly how the project must be taught. A student who receives a complete 150-line program has learned to copy, not to engineer.

Session 1

Understand the problem discuss it, sketch the block diagram and algorithm, no code yet.

Session 2

Build the basic hardware assemble and wire, with no logic beyond simple on/off tests.

Session 3

Test individual components each sensor/actuator alone, exactly as in the Level references.

Session 4

Write the first simple program Version 1, the smallest working piece.

Session 5

Integrate components build up to the Final Version incrementally.

Session 6

Debug and improve apply the Debugging Guide, then attempt the Student Challenges.

Appendix A The Debugging Guide

"It doesn't work" is not a diagnosis.

Every debugging session in this course starts with one question: What exactly is not working? Not 'the robot is broken' but 'the robot moves forward correctly but never stops when close to a wall.' A precise symptom points to a small number of possible causes. A vague symptom points to everything, which is the same as pointing to nothing.

The Debugging Method

- Describe the exact symptom. What does the system do? What should it do instead? When exactly does the difference appear?
- Isolate the section responsible. Which part of SENSE – THINK – ACT is failing is the sensor reading wrong, is the decision wrong, or is the actuator not responding to a correct decision?
- Test that section alone. Remove or bypass everything else. Print raw sensor values. Trigger the actuator directly with a fixed value, without the decision logic.
- Form a hypothesis. Based on what you found, what do you think is actually wrong?
- Test the hypothesis with the smallest possible change. Change one thing. Re-test. Do not change five things at once you will not know which change fixed (or broke) anything.
- Confirm the fix, then re-test the whole system. A fix to one part sometimes reveals or causes a problem elsewhere.

Categories of Error

[illegible]

Syntax errors

Code fails to compile at all; the IDE reports a line number and an error.

Read the exact error message and line number

the fix is almost always exactly there or on the line just above it (a missing semicolon or brace).

Wrong pin numbers

Code compiles and runs, but the wrong component reacts, or nothing reacts.

Physically trace the wire from the component to the board and compare against the pin number in code.

Incorrect wiring

Component behaves randomly, not at all, or is damaged.

Recheck against the wiring diagram; verify VCC/GND are not swapped; verify a common GND exists across all power sources.

Wrong sensor thresholds

Logic runs, but decisions seem wrong (e.g. light never turns on, or always on).

Print the raw sensor value to Serial Monitor in the actual test conditions and compare it to the threshold in code.

Logic errors

Code runs without crashing, but does the wrong thing.

Trace the code by hand, line by line, with a specific example value, exactly as if you were the computer.

Motor direction problems

A motor spins the wrong way, or a robot turns instead of going straight.

Swap the two direction pins for that motor in code (not the wiring) and re-test.

Power problems

Board resets randomly, especially when motors/pumps start; Wi-Fi drops during transmission.

Check the power supply can deliver enough current for every component running at once, and that grounds are common across supplies.

Library problems

Compile errors mentioning a library name; sensor readings are nonsensical (e.g. NaN).

Confirm the correct library is installed (exact name and, if relevant, version) and that #include matches it exactly.

Questions to Ask Instead of "It Doesn't Work"

1. What exactly did you expect to happen, and what actually happened?
2. Does the problem happen every time, or only sometimes? What's different between the times it works and the times it doesn't?
3. Have you tested this part of the system on its own, separate from everything else?
4. What is the raw sensor value right now, in this exact condition have you actually looked, or are you assuming?
5. What is the very last thing that changed before this stopped working?

Appendix C Master Component Reference & Reuse Plan

This course is deliberately designed so that most components are reused across multiple projects, minimising cost. The table below shows every major component used in this manual, and every project it appears in.

[illegible]

Arduino Uno (or compatible)
 Level 7
 Every Arduino-based project (Projects 1-7)
 ESP32 dev board
 Phase 6
 Project 8, ESP32 IoT Smart Farm Monitor, Capstone (optional)
 LED + 220Ω resistor
 Project 1

Level 8, Project 4 (status LED), Project 8
 Push button
 Level 8
 Project 5 (manual feed), Project 8 (arm/disarm)
 Potentiometer
 Level 8
 Level 9 (motor speed practice), general testing substitute for any analog sensor
 LDR + 10kΩ resistor
 Level 8
 Project 8 (automatic lighting)
 HC-SR04 ultrasonic sensor
 Level 8
 Projects 3, 6, 7
 Servo motor
 Level 9
 Projects 3, 5, 6, 7
 Buzzer
 Level 9
 Projects 5, 8
 L298N motor driver + DC geared motors
 Level 9 / Level 10
 Projects 2, 3, 6, 7
 Relay module
 Level 9
 Projects 4, 8
 2WD chassis, wheels, caster
 Level 10
 Projects 2, 3, 6
 HC-05 Bluetooth module
 Project 2
 Optional override in Projects 3, 6, 7
 Capacitive soil moisture sensor
 Level 8
 Project 4, ESP32 IoT Smart Farm Monitor
 DHT11/DHT22 sensor
 Level 8
 ESP32 IoT Smart Farm Monitor
 DS3231 RTC module
 Project 5
 Capstone (optional, any time-based system)
 OLED/LCD display
 Project 4 (optional)
 Project 5 (optional), Project 8, ESP32 IoT Smart Farm Monitor
 PIR motion sensor
 Project 8
 Capstone (optional, security-themed projects)

Procurement note for schools

A school equipping one class of ~6 groups needs roughly: 6× Arduino Uno, 2-3× ESP32 (can be shared/rotated for Phase 6), 6× basic sensor/actuator kits (LED, button, potentiometer, LDR, buzzer, servo), 6× HC-SR04, 6× L298N + 2 DC motors + chassis kits, 1-2× HC-05, 6× soil moisture + DHT sensors, 1-2× DS3231 RTC (can rotate), 1-2× PIR + OLED sets. Components are intentionally reused rather than bought fresh for every project.

Appendix D Glossary

[illegible]

Actuator

A device that converts an electrical signal into physical action e.g. a motor, servo, buzzer, or pump.

Algorithm

A logical, ordered sequence of steps to solve a problem, independent of any programming language.

Array

A data structure that stores multiple values of the same type under one name, accessed by index (starting at 0).

Actuator driver / Motor driver

A circuit (e.g. L298N) that safely lets a low-power microcontroller pin control a high-current motor.

Analog signal

A signal that can take a continuous range of values (e.g. 0-1023 via `analogRead()`), unlike a digital HIGH/LOW.

Baud rate

The speed of serial communication, measured in bits per second both ends of a serial link must match.

Calibration

The process of testing a sensor in real conditions to choose an accurate threshold or scaling value.

Debugging

The systematic process of finding and fixing the cause of unexpected behaviour in a system.

Digital signal

A signal with only two states: HIGH (on) or LOW (off).

Flowchart

A diagram representing an algorithm using standard shapes: oval (start/end), rectangle (action), diamond (decision).

Function

A named, reusable block of code that performs one clear task, optionally taking parameters and returning a value.

GPIO

General Purpose Input/Output a pin on a microcontroller that can be configured as digital input or output.

Hardware

The physical, touchable parts of a system: boards, sensors, motors, wires.

IoT (Internet of Things)

Physical devices with sensors/actuators that connect to the internet to send or receive data.

Loop

A block of code that repeats `for`, `while`, and `do-while` in C++; `loop()` itself repeats forever on Arduino/ESP32.

Microcontroller

A small computer on a single chip (CPU + memory + I/O) that runs one program to control a physical device.

Parameter

A value passed into a function so it can behave differently each time it is called.

PWM (Pulse Width Modulation)

A technique of rapidly switching a pin on/off to simulate a variable analog output, used for motor speed and brightness.

Sensor

A device that measures a physical quantity (light, distance, moisture, motion, etc.) and converts it into an electrical signal.

Serial communication

Sending data one bit at a time over a wire (or wirelessly, e.g. Bluetooth) between two devices.

SENSE – THINK – ACT

The central mental model of this course: a sensor senses, code decides, an actuator acts.

Software

The instructions (code) that tell hardware what to do.

Threshold

A chosen value against which a sensor reading is compared to make a decision (e.g. 'dark' vs 'bright').

Variable

A named location in memory that holds a value which can change while a program runs.

Wi-Fi / BLE

Wireless communication standards; ESP32 supports both, enabling IoT and wireless-control projects.

ENHANCED EDITION FOR ONLINE AND CLASSROOM DELIVERY

This comprehensive manual is designed for delivery at Algorise Tech Explorers (ATE) and is available alongside our hosted academy platform at www.muwatta.com.ng. All students have access to video demonstrations, interactive coding environments, and progress tracking through the website.

IMPORTANT FOR ABSOLUTE BEGINNERS: You do not need any prior programming experience. You do not need to have touched an Arduino before. You do not need to understand electricity. If you can follow instructions carefully and ask questions when confused, you can complete this course. That is all that is required.